

UE24CS352A: Machine Learning

Dr. Shylaja S S

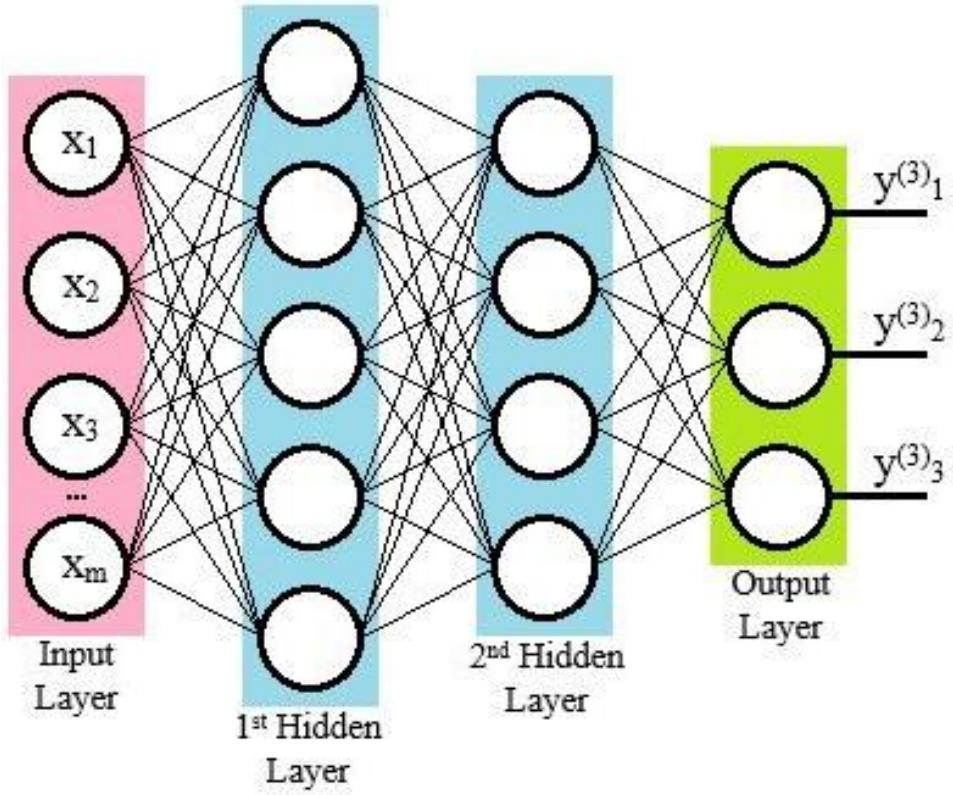
Professor

Department of Computer Science & Engineering

Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

Multilayer network

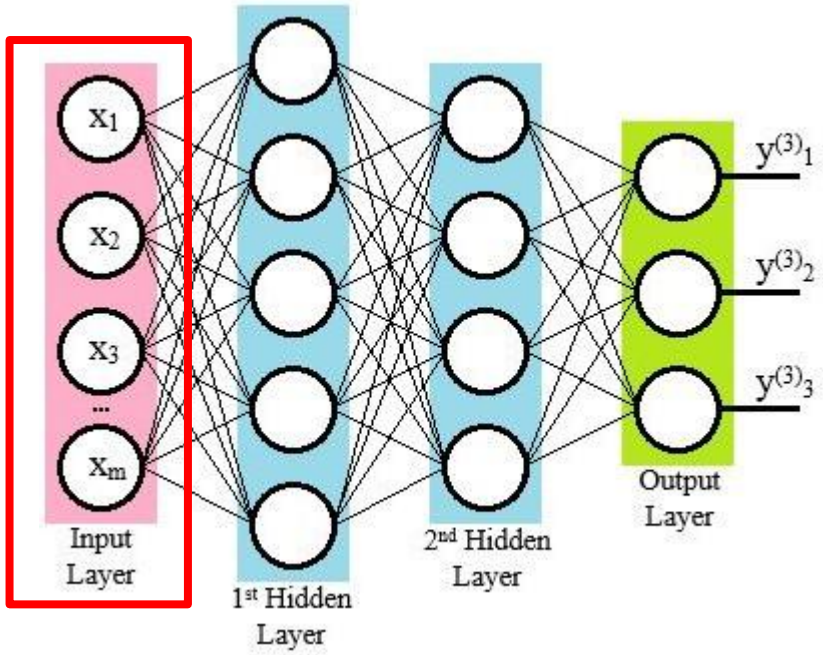
- We previously saw how a single neuron can't be used to learn complex functions.
- This is the reason our brain has billions of neuron, stacked in layers forming a network.
- Similarly, artificial neurons are arranged in layers.
- Each layer will be connected in such a way that information is passed from one layer to another.



Multilayer network

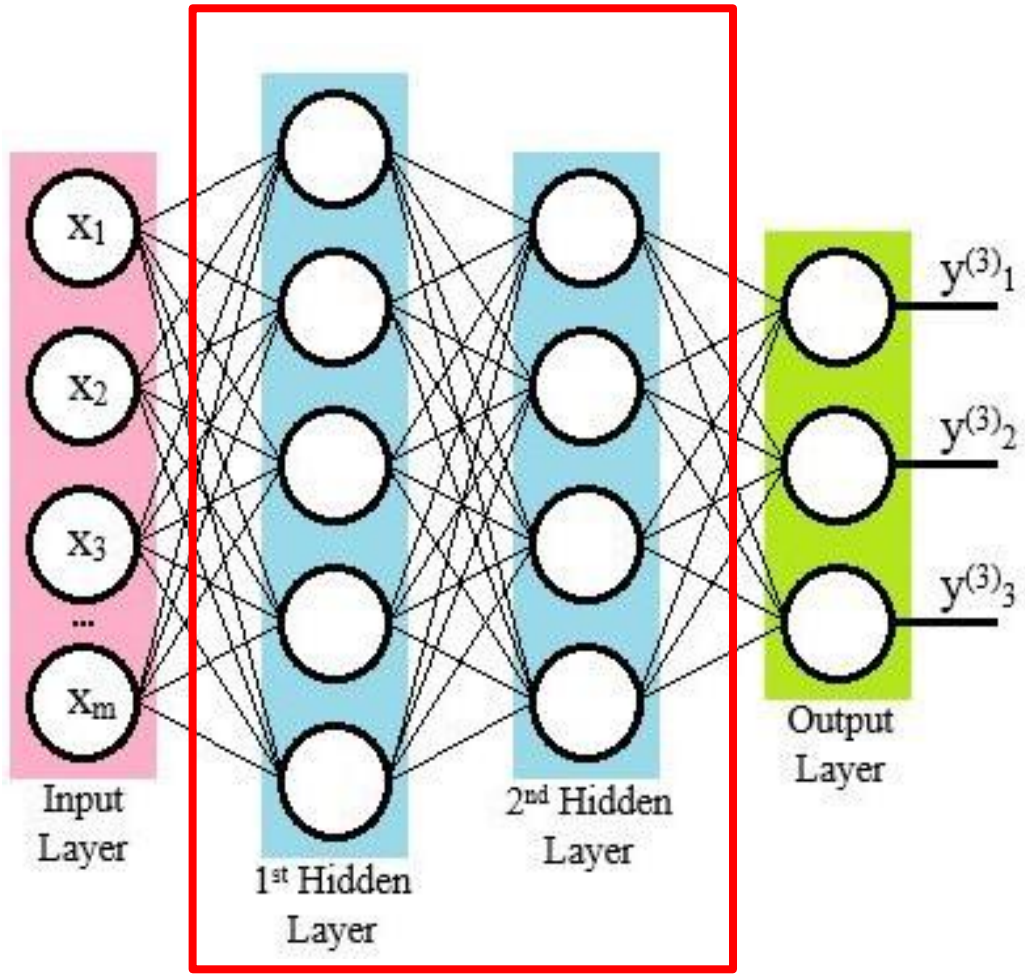
- A typical ANN consists of the input layer, hidden layer(s) and output layer.
- If it has more than 1 hidden layer, it is called a deep ANN (DNN).
- A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).
- Each layer has a collection of neurons, and the neurons in a layer interact with all neurons in other layers.
- We use the term nodes or units to represent the neurons in an ANN.
- The number of layers and the number of neurons are referred to as hyperparameters of a neural network.

Input layer



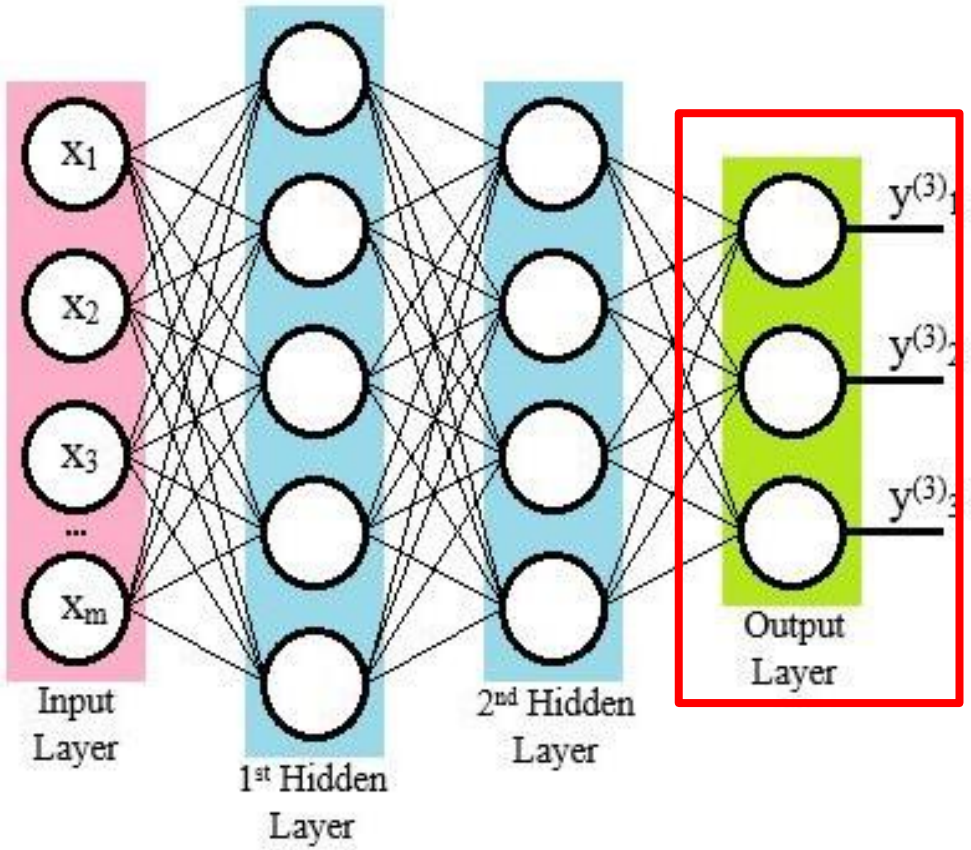
- The input layer is where we feed in inputs to the network.
- The number of units in input layer is the number of inputs we feed to the network.
- No computation is performed in the input layer

Hidden layer(s)



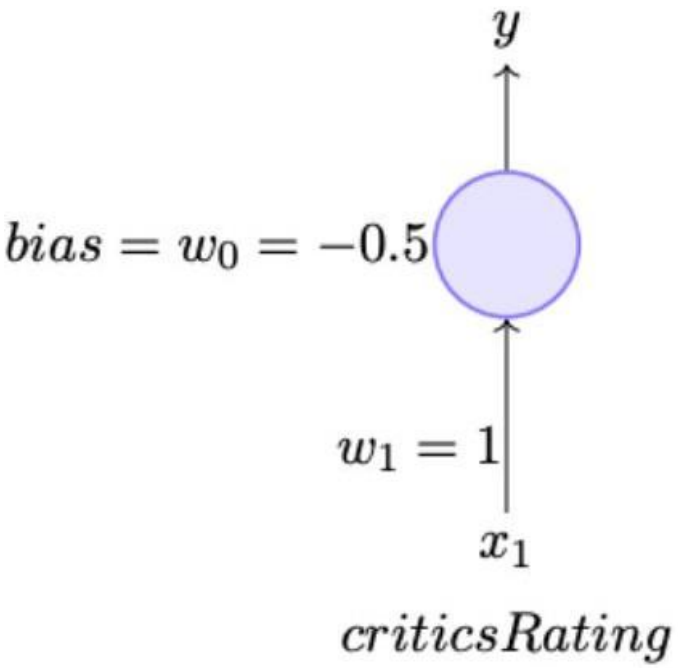
- Any layer between the input layer and the output layer is called hidden layers.
- It is the main core for learning, processes the input received from input layer.
- The hidden layer is responsible for deriving complex relationships between input and output. It identifies patterns in the dataset.
- There can be n number of hidden layer according to our use case.

Output layer



- After processing the input the hidden layer sends its results to the output layer.
- As the name suggests the output layer emits output

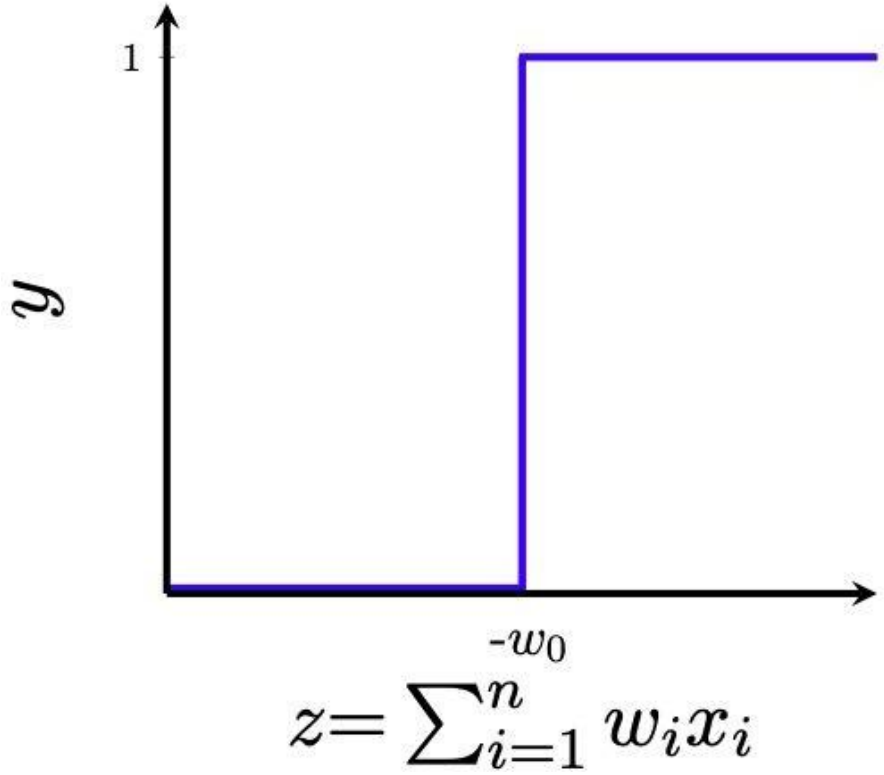
Perceptron thresholding logic is harsh



- The thresholding logic used by a perceptron is very harsh !
- For example, let us return to our problem of deciding whether we will like or dislike a movie
- Consider that we base our decision only on one input ($x_1 = criticsRating$ which lies between 0 and 1)
- If the threshold is 0.5 ($w_0 = -0.5$) and $w_1 = 1$ then what would be the decision for a movie with $criticsRating = 0.51$? (like)
- What about a movie with $criticsRating = 0.49$? (dislike)
- It seems harsh that we would like a movie with rating 0.51 but not one with a rating of 0.49

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

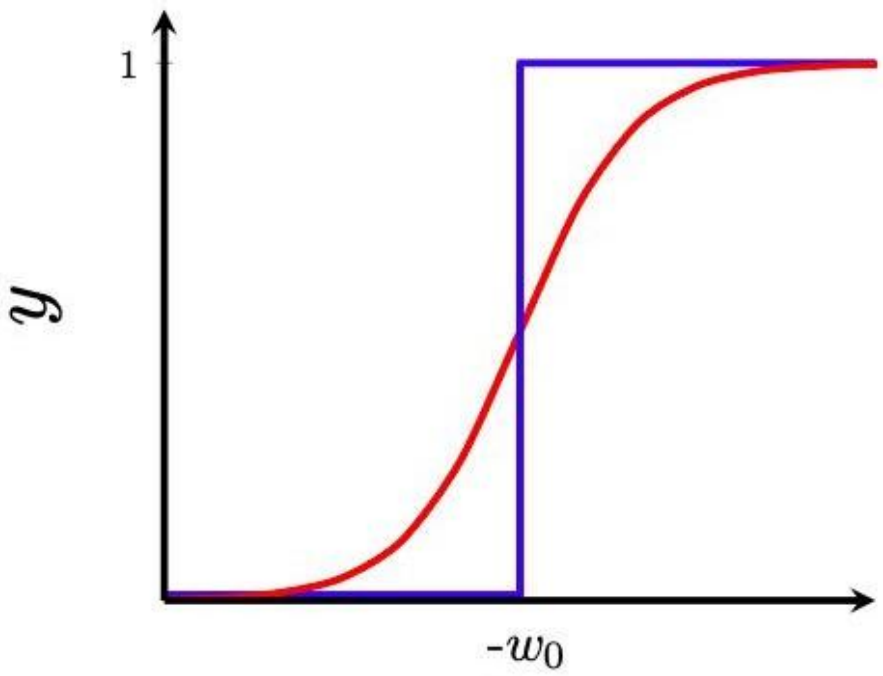
Perceptron thresholding logic is harsh



- This behavior is not a characteristic of the specific problem we chose or the specific weight and threshold that we chose
- It is a characteristic of the perceptron function itself which behaves like a step function
- There will always be this sudden change in the decision (from 0 to 1) when $\sum_{i=1}^{n} w_i x_i$ crosses the threshold ($-w_0$)
- For most real world applications we would expect a smoother decision function which gradually changes from 0 to 1

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Sigmoid Neuron



- Introducing sigmoid neurons where the output function is much smoother than the step function
- Here is one form of the sigmoid function called the logistic function

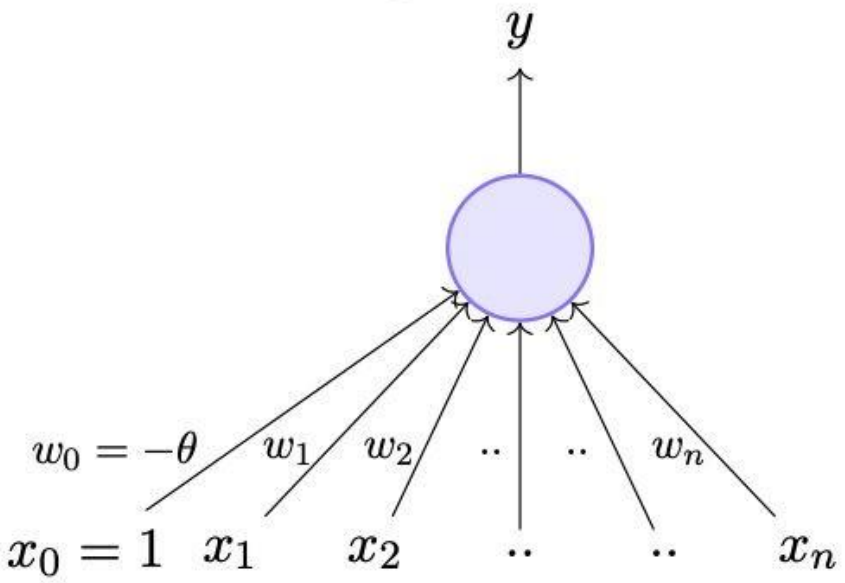
$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^{n} w_i x_i)}}$$

- We no longer see a sharp transition around the threshold -w_0
- Also the output y is no longer binary but a real value between 0 and 1 which can be interpreted as a probability
- Instead of a like/dislike decision we get the probability of liking the movie

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Perceptron vs Sigmoid Neuron

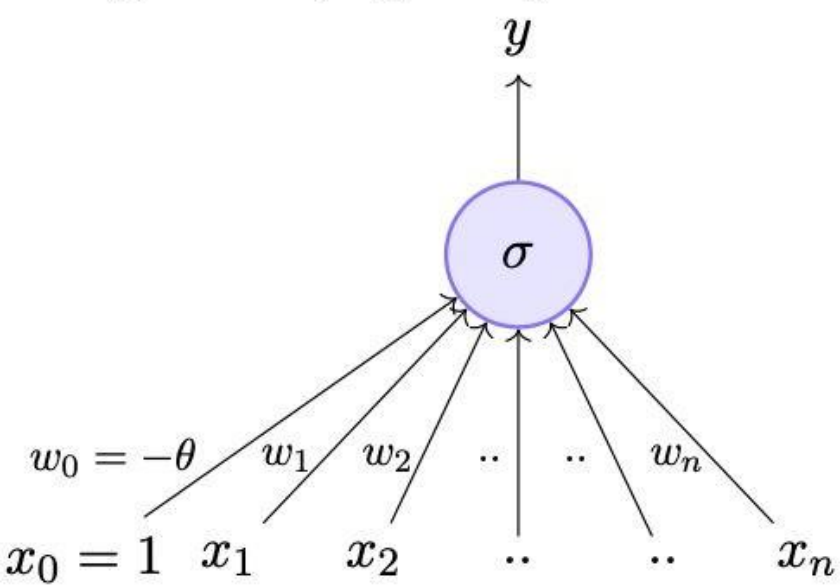
Perceptron



$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$= 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

Sigmoid (logistic) Neuron

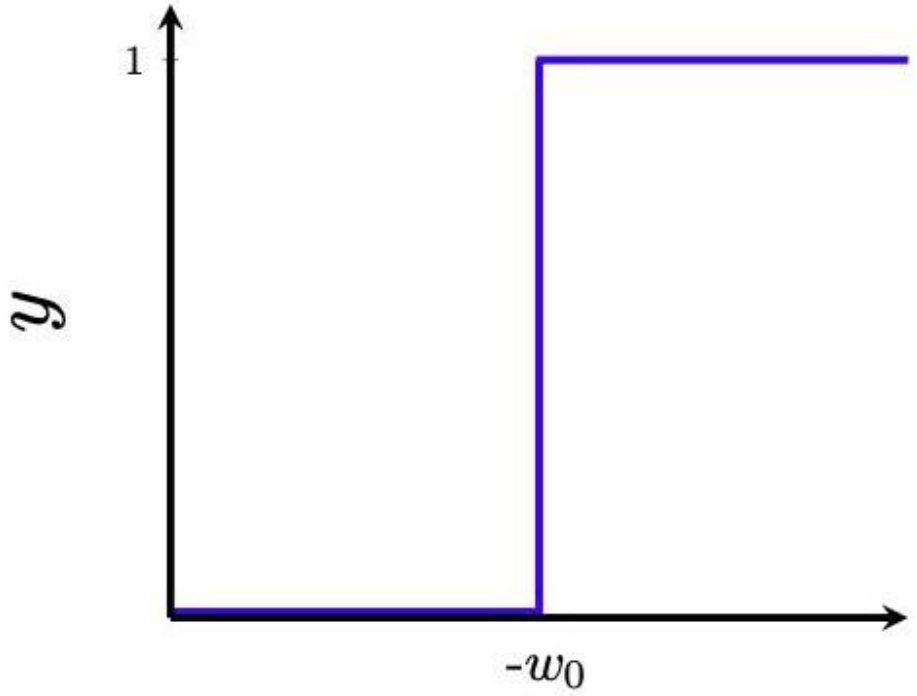


$$y = \frac{1}{1 + e^{-(\sum_{i=0}^{n} w_i x_i)}}$$

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Perceptron vs Sigmoid Neuron

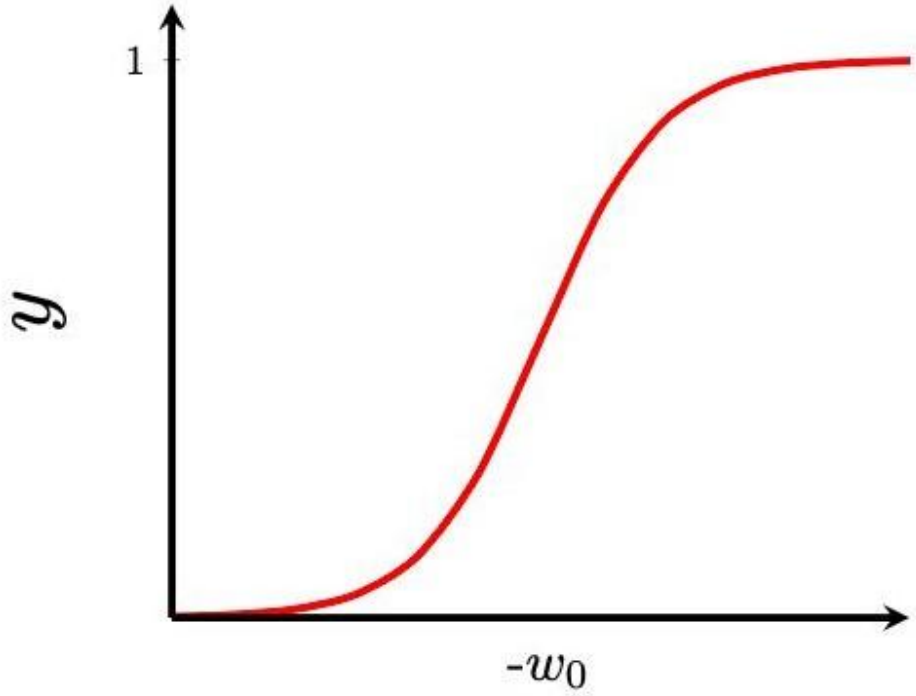
Perceptron



$$z = \sum_{i=1}^{n} w_i x_i$$

Not smooth, not continuous (at $w0$), **not differentiable**

Sigmoid Neuron

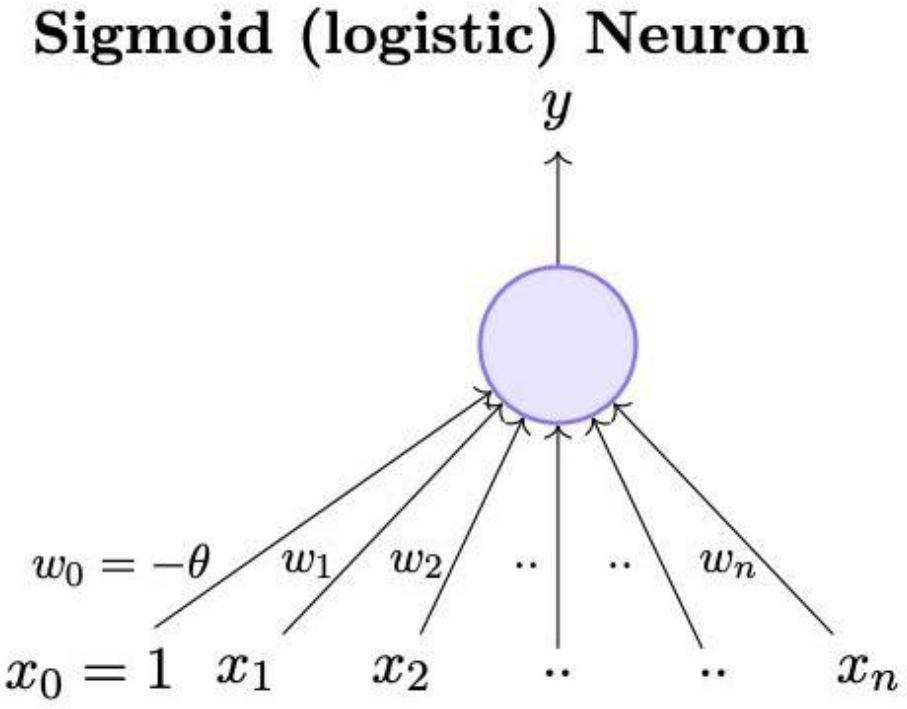


$$z = \sum_{i=1}^{n} w_i x_i$$

Smooth, continuous, **differentiable**

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

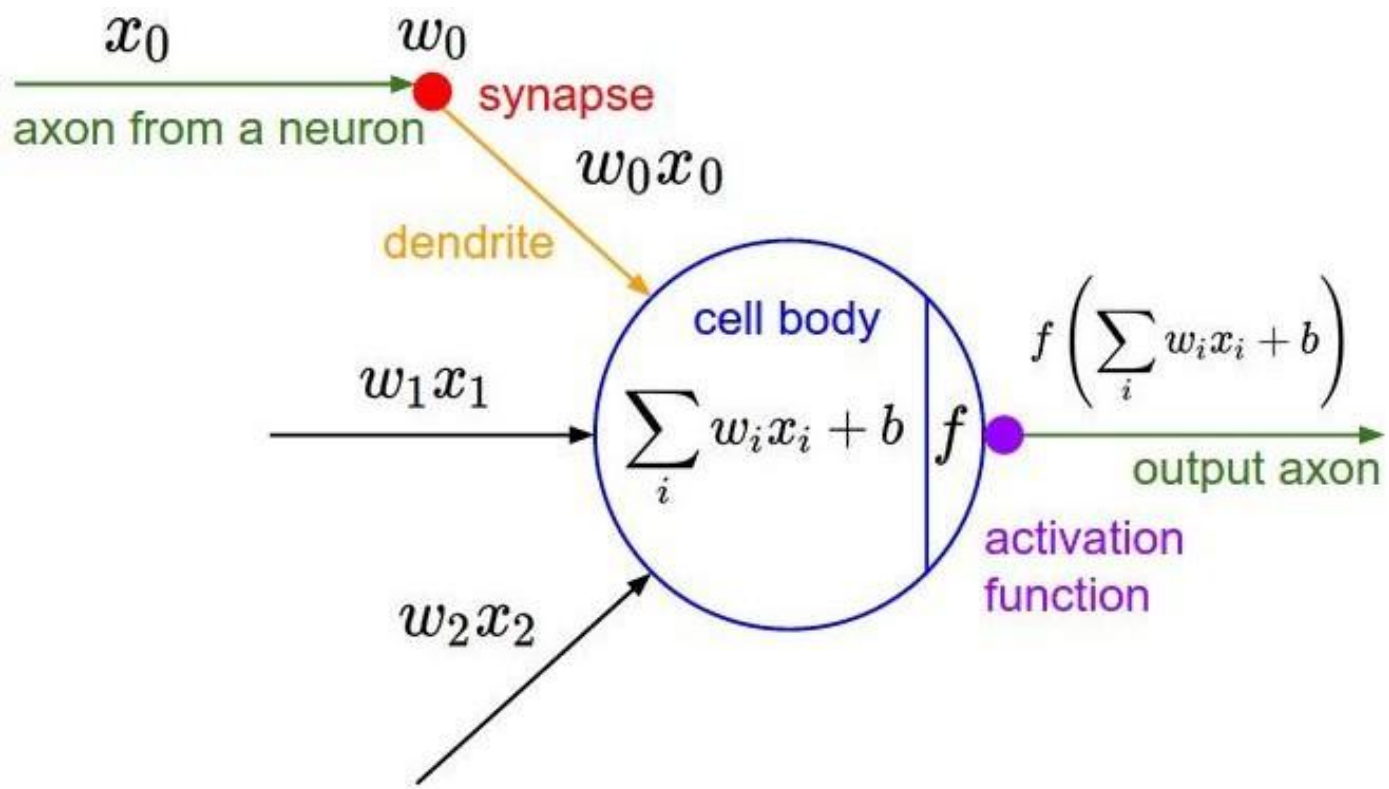
Learning weights when output is real-valued



- Well, just as we had an algorithm for learning the weights of a perceptron, we also need a way of learning the weights of a sigmoid neuron

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Artificial neuron in an Artificial Neural Network



The most fundamental unit of an ANN

Objective function/Cost function/Loss function/Error function

- The search for weights is driven by the objective of minimizing the error between prediction and true value.
- Because the error can be positive or negative, most often we take the square.
- Depending on the problem, suitable loss function can be chosen.
- Example :

a) Mean squared error, $$\mathbf{MSE} = \frac{1}{2}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

b) Cross Entropy - measures the performance of a classification model whose output is a probability value between 0 and 1.

$$L = -\sum_{i=1}^{N} y^{(i)} * \log \hat{y}^{(i)}$$

Forward Propagation

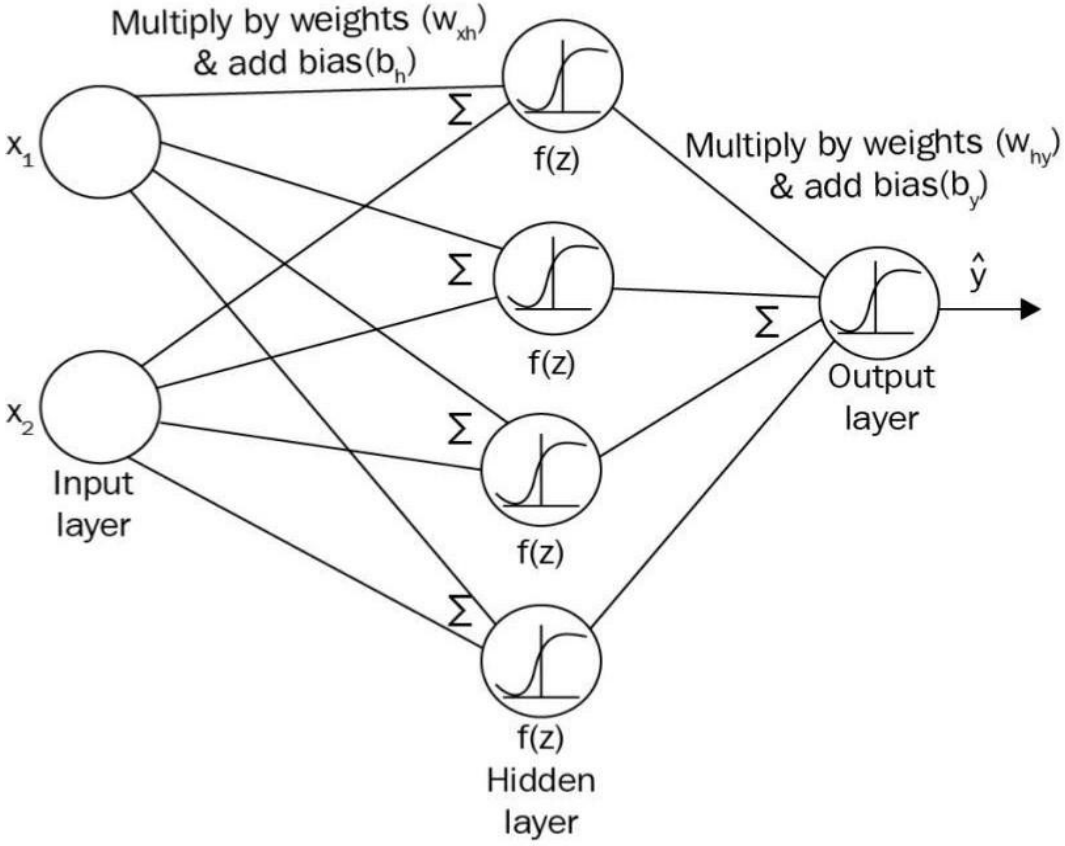
Process of moving from input to output layer, computing net sum and output at each neuron

Forward propagation

What will the dimension of the weight matrix be?

Number of units in the previous layer
X
Number of units in the current layer

- We first need to initialize the weight matrix.
- In the real world we don't know which inputs are relatively important hence we randomly initialize weights and biases.
- The inputs are then multiplied by weights, bias is added the result is propagated to the hidden layer where the activation function will be applied.



Notations used

x_{ji} = the ith input to unit j

w_{ji} = the weight associated with the ith input to unit j

$net_j = \sum_i w_{ji} X_{ji}$ (the weighted sum of inputs for unit j)

o_j = the output computed by unit j

t_j = the target output for unit j

Applicable only to Neurons in Output Layer

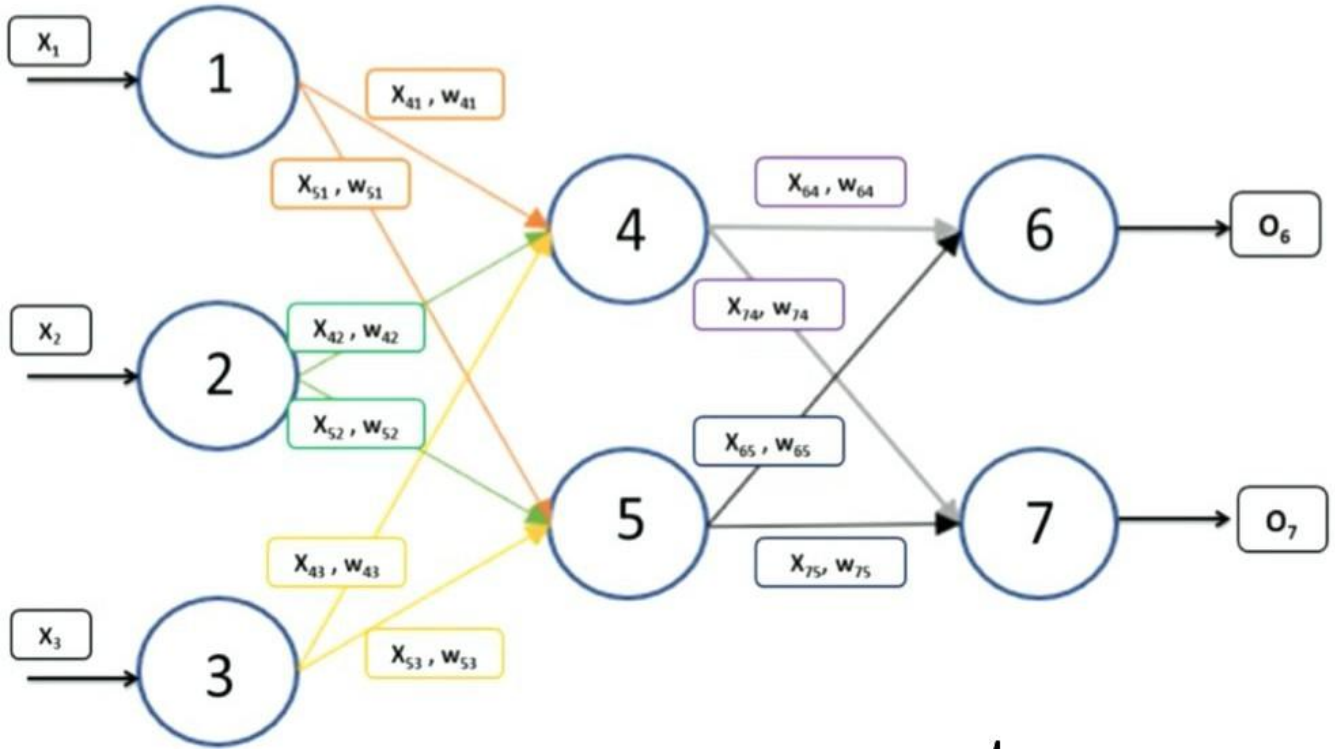
σ = the sigmoid function

outputs = the set of units in the final layer of the network

Downstream(j) = the set of units whose immediate inputs include the output of unit j

Applicable only to Neurons in Hidden Layer

Calculation of a weight matrix



Downstream(4) = {6,7}
Downstream(5) = {6,7}

outputs = {6, 7}

Weight Matrix Shape

#neurons in previous layer X #neurons in current layer

$$W_1 = \begin{bmatrix} w_{41} & w_{51} \\ w_{42} & w_{52} \\ w_{43} & w_{53} \end{bmatrix}_{3\times 2}$$

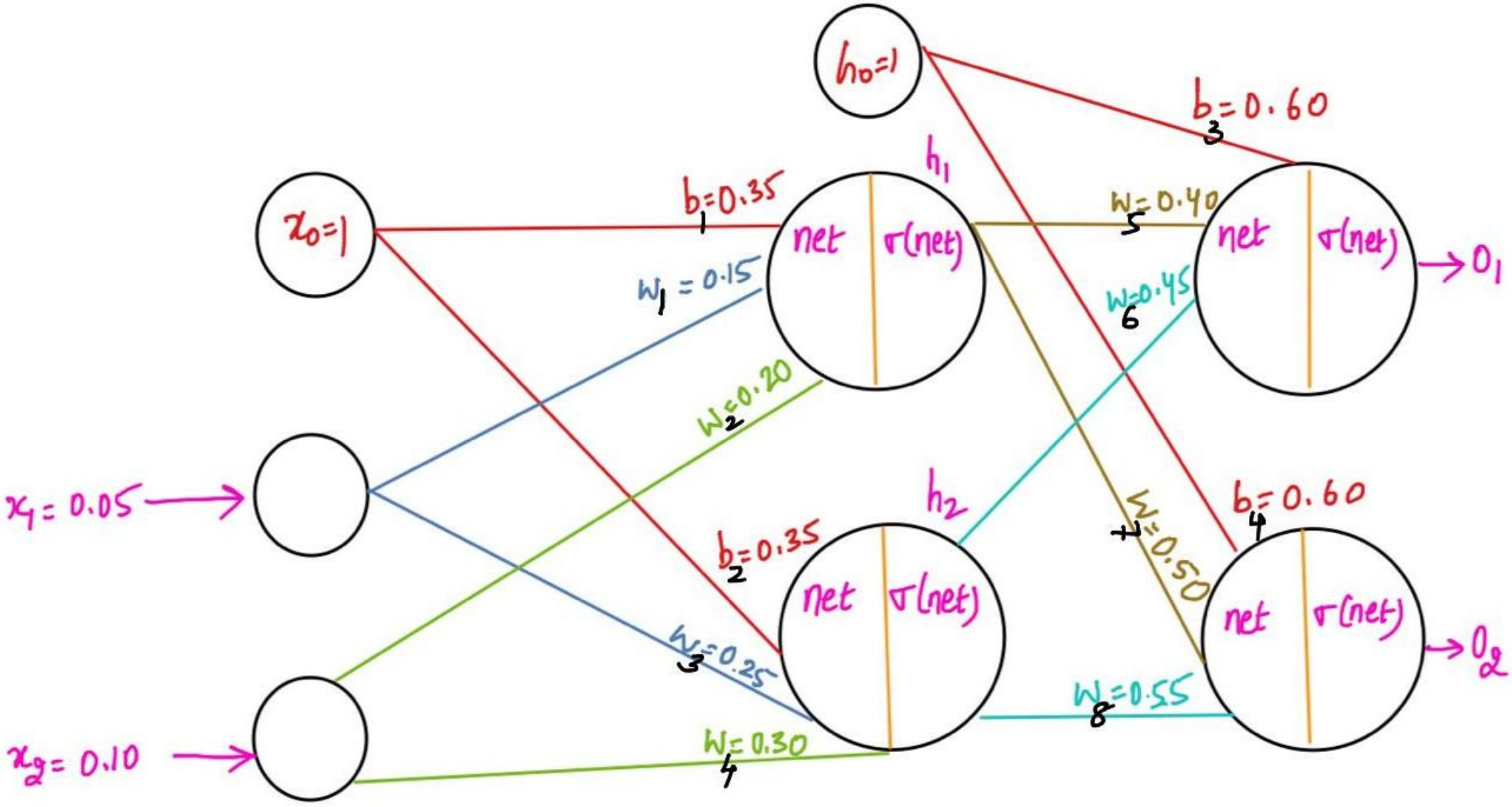
w_{ji}

j: current layer neuron number

i: previous layer neuron number

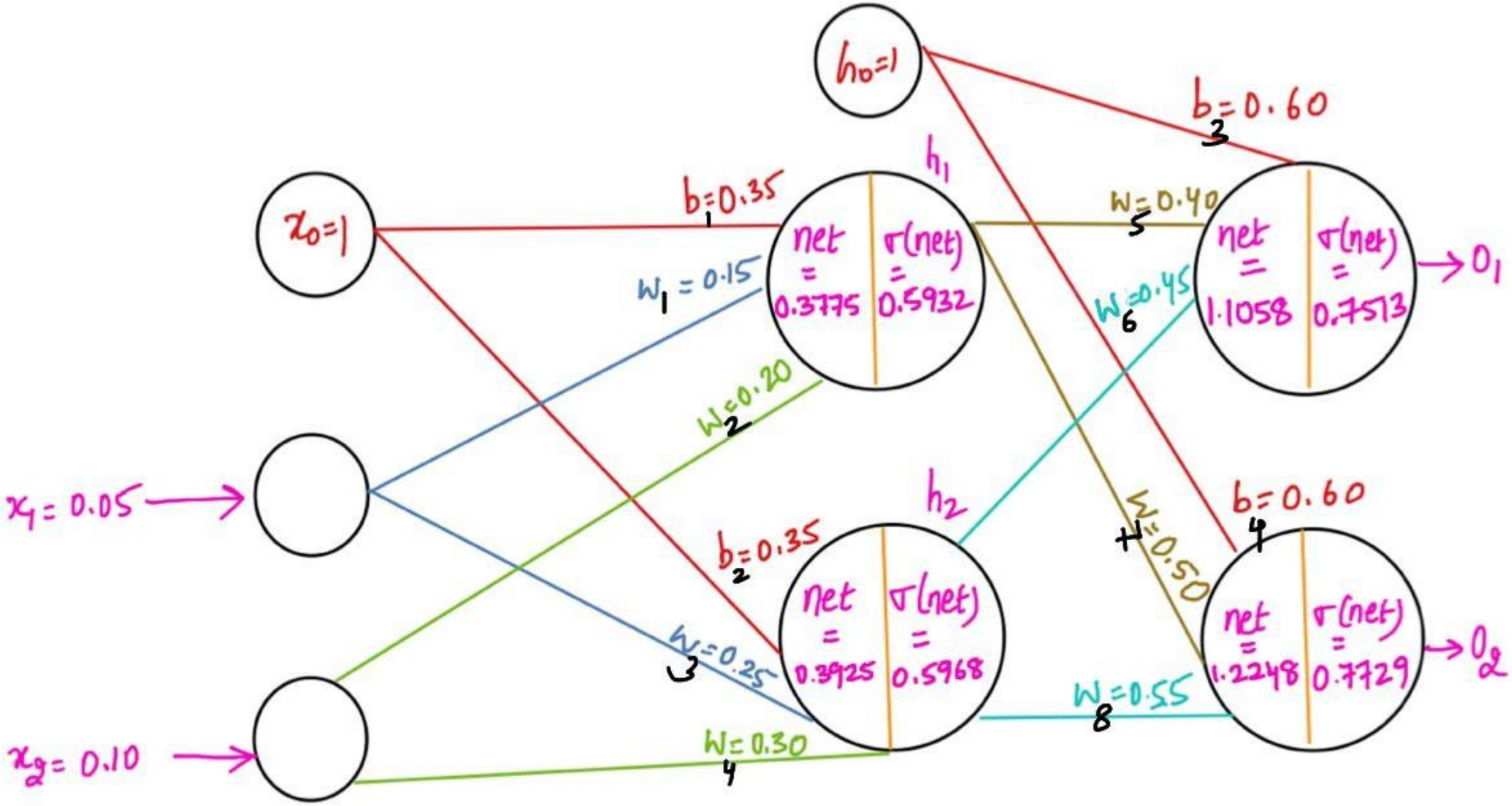
Problem 1

Forward propagation --- Problem 1



t(target values at output neuron)
0.01
0.99

Forward propagation Problem 1--- Solution



Forward propagation Problem 1--- Solution

Output calculation at hidden layer

$$Z_1 = (W1)^T * X + b$$

Z_1: net sum; $W1$: Weight Matrix; X: Input vector; b: Bias vector

$$\begin{bmatrix} net_1 \\ net_2 \end{bmatrix} = \begin{bmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{bmatrix} * \begin{bmatrix} 0.05 \\ 0.10 \end{bmatrix} + \begin{bmatrix} 0.35 \\ 0.35 \end{bmatrix}$$

$$\begin{bmatrix} net_1 \\ net_2 \end{bmatrix} = \begin{bmatrix} 0.3775 \\ 0.3925 \end{bmatrix}$$

$$\alpha_1 = \sigma(Z_1) = \begin{bmatrix} \sigma(net_1) \\ \sigma(net_2) \end{bmatrix} = \begin{bmatrix} 0.5932 \\ 0.5968 \end{bmatrix}$$

α_1: output of activation function

Output Calculation at output layer

$$Z_2 = (W2)^T * \alpha_1 + b$$

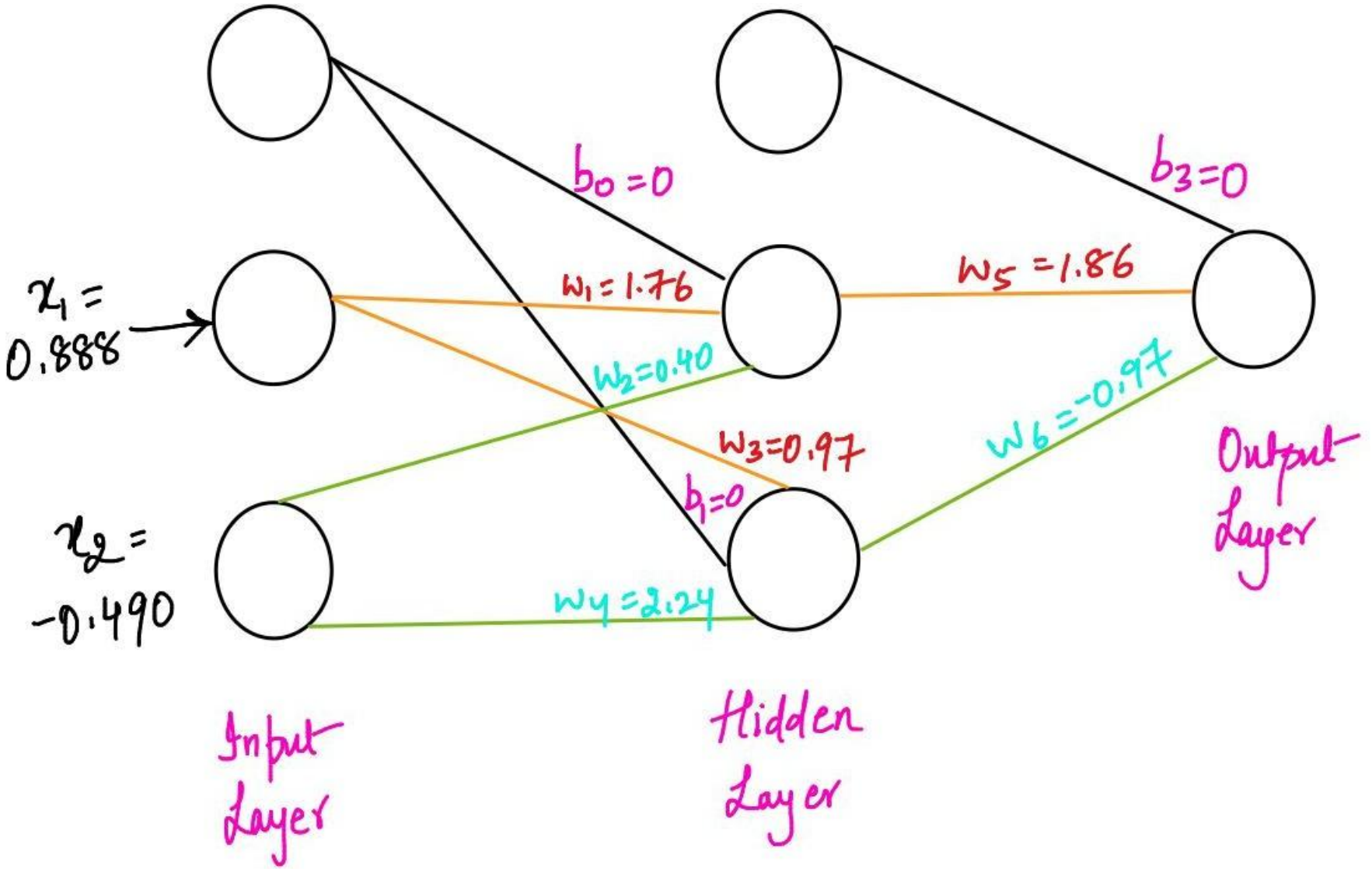
$$\begin{bmatrix} net_1 \\ net_2 \end{bmatrix} = \begin{bmatrix} 0.40 & 0.45 \\ 0.50 & 0.55 \end{bmatrix} * \begin{bmatrix} 0.5932 \\ 0.5968 \end{bmatrix} + \begin{bmatrix} 0.60 \\ 0.60 \end{bmatrix}$$

$$\begin{bmatrix} net_1 \\ net_2 \end{bmatrix} = \begin{bmatrix} 1.1058 \\ 1.2248 \end{bmatrix}$$

$$\hat{y} = \sigma(Z_2) = \begin{bmatrix} \sigma(net_1) \\ \sigma(net_2) \end{bmatrix} = \begin{bmatrix} 0.7513 \\ 0.7729 \end{bmatrix}$$

Problem 2

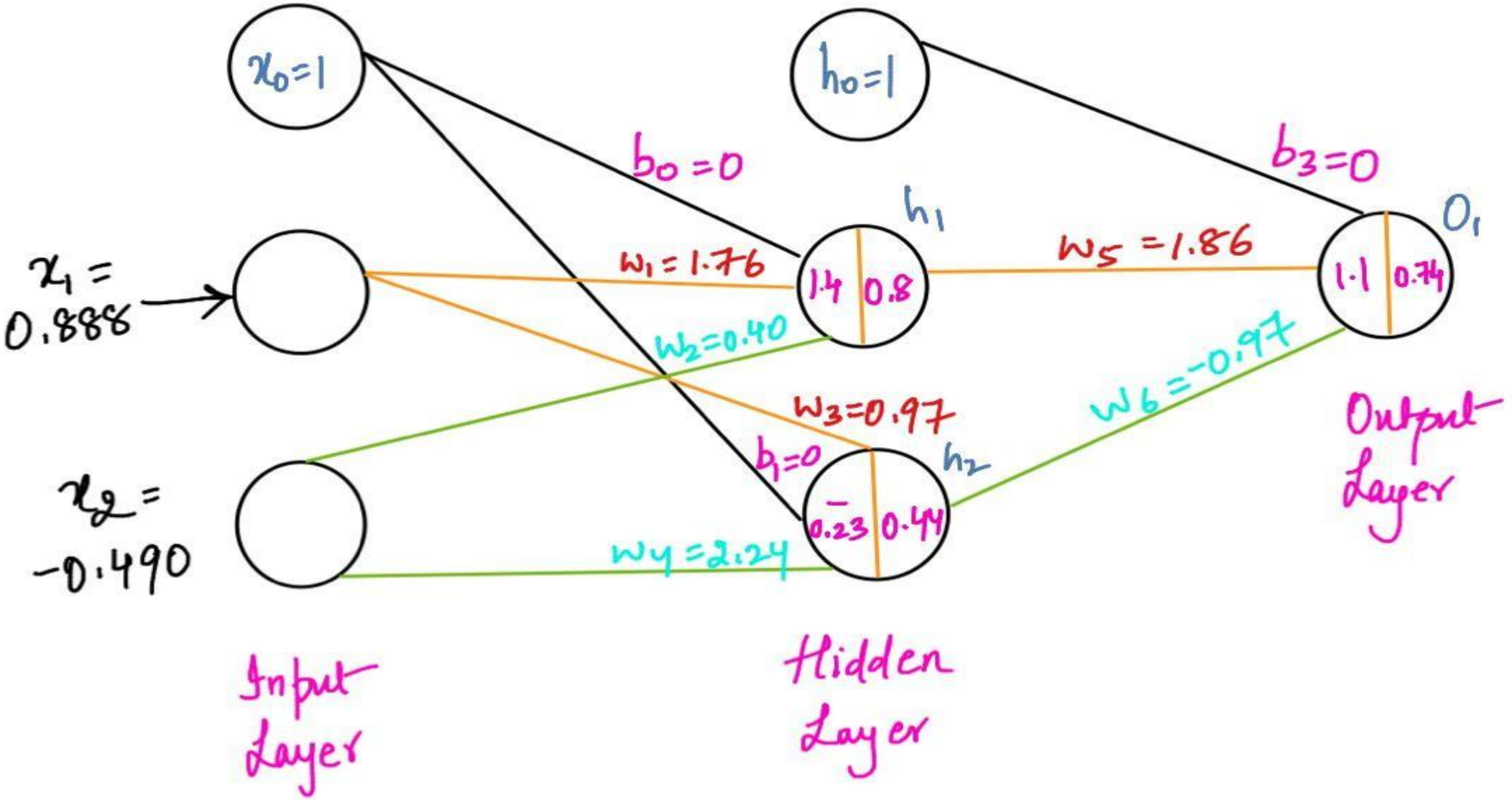
Forward propagation Problem - 2



Forward propagation Problem 2 - Solution

- During forward propagation at each node of hidden and output layer pre-activation and activation takes place.
- At the first node of the hidden layer, **a1**(pre-activation) is calculated first and then **h1**(activation) is calculated.
- **a1** is a weighted sum of inputs. Here, the weights are randomly generated.
- **a1** = w1*x1 + w2*x2 + b1 = 1.76* 0.88 + 0.40*(-0.49) + 0 = 1.37
- **h1** is the value of activation function applied on a1. $h1 = \frac{1}{1+e^{-a1}}$ = 0.8 approx.
- **a2** = w3*x1 + w4*x2 + b2 = 0.97 *0.88 + 2.24 *(- 0.49)+ 0 = -2.29 approx. $h2 = \frac{1}{1+e^{-a2}}$ = 0.44 approx.
- For any layer after the first hidden layer, the input is output from the previous layer.
- **a3** = w5*h1 + w6*h2 + b3 = 1.86*0.8 + (-0.97)*0.44 + 0 = 1.1 approx. $h3 = \frac{1}{1+e^{-a3}}$ = 0.74 approx.

Forward propagation Problem 2 - Solution



Gradient Descent

Goal : To find a better way of traversing the error surface so that we can reach the minimum value quickly without resorting to brute force search!

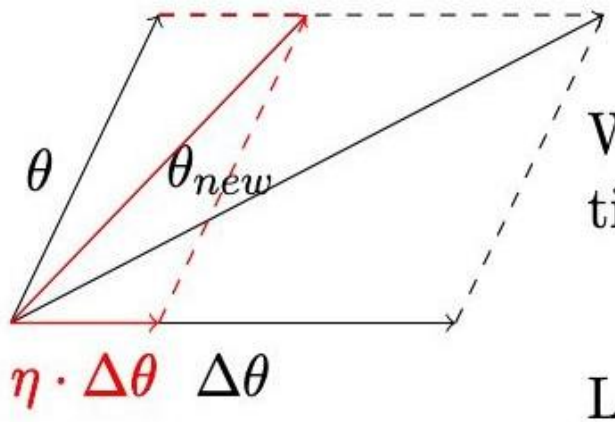
Gradient Descent

vector of parameters, say, randomly initialized

$\theta = [w, b]$

$\Delta\theta = [\Delta w, \Delta b]$

change in the values of w, b



We moved in the direction of $\Delta\theta$

Let us be a bit conservative: move only by a small amount η

$$\theta_{new} = \theta + \eta \cdot \Delta\theta$$

Question: What is the right $\Delta\theta$ to use?

The answer comes from Taylor series

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Taylor series --- Recap

- Used to evaluate/approximate the value of a function at any point
- Taylor series of a function is an infinite sum of terms that are expressed in terms of the function's derivatives at a single point.
- Let’s say we have a function with only 1 parameter, f(x). Assume at a given point x, we want to evaluate the function at x+h, where h is a small step in the direction of x.
- We can use Taylor series to evaluate the value of function at x+h point.

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(x) + \cdots$$

Taylor series --- Recap

- Taylor formula for function of two variables : f(x,y)

$$f(x+a, y+b) = f(x,y) + \frac{a^1}{1!_0}\frac{\partial f(x,y)}{\partial x} + \frac{b^1}{1!_0}\frac{\partial f(x,y)}{\partial y} + \frac{a^2}{2!_0}\frac{\partial^2 f(x,y)}{\partial x^2} + \frac{b^2}{2!_0}\frac{\partial^2 f(x,y)}{\partial y^2} + ab\frac{\partial^2 f(x,y)}{\partial x \partial y}$$

- We can also use vector notation to represent two-variable Taylor series as:

$$f(\vec{x}+\vec{u}) = f(\vec{x}) + (\vec{u})^T \cdot \nabla f(\vec{x}) + \frac{1}{2}(\vec{u})^T \cdot H_f(\vec{x}) \cdot \vec{u}$$

$$\nabla f(\vec{x}) = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

gradient of f

$$H_f(\vec{x}) = \begin{bmatrix} \frac{\partial^2 f(x,y)}{\partial x^2} & \frac{\partial^2 f(x,y)}{\partial x \partial y} \\ \frac{\partial^2 f(x,y)}{\partial y \partial x} & \frac{\partial^2 f(x,y)}{\partial y^2} \end{bmatrix}$$

Hessian matrix ($\nabla^2 f(\vec{x})$)

Let $\vec{x} = \begin{bmatrix} x \\ y \end{bmatrix}$, $\vec{u} = \begin{bmatrix} a \\ b \end{bmatrix}$

$\vec{x}$ ↙ input vector

$\vec{u}$ ↙ Change in values

Gradient Descent

For our purpose, the function we want to approximate is a loss function.

For ease of notation, let $\Delta\theta = u$, then from Taylor series, we have,

$$\begin{aligned}\mathscr{L}(\theta+\eta u) &= \mathscr{L}(\theta) + \eta * u^T \nabla \mathscr{L}(\theta) + \frac{\eta^2}{2!} * u^T \nabla^2 \mathscr{L}(\theta) u + \frac{\eta^3}{3!} * \ldots + \frac{\eta^4}{4!} * \ldots \\ &= \mathscr{L}(\theta) + \eta * u^T \nabla \mathscr{L}(\theta) \ [\eta \textit{ is typically small, so } \eta^2, \eta^3, \ldots \to 0]\end{aligned}$$

$$\mathscr{L}(\theta+\eta u) - \mathscr{L}(\theta) = u^T \nabla \mathscr{L}(\theta)$$

Note that the move (ηu) would be favorable only if,

$$\mathscr{L}(\theta+\eta u) - \mathscr{L}(\theta) < 0 \ [\textit{i.e., if the new loss is less than the previous loss}]$$

This implies, $\boxed{u^T \nabla \mathscr{L}(\theta) < 0}$

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Gradient Descent

Let β be the angle between u^T and $\nabla \mathscr{L}(\theta)$, then we know that,

$$\boxed{u^T \nabla \mathscr{L}(\theta) < 0}$$

This implies, $k * cos(\beta) < 0$

Thus, $\mathscr{L}(\theta + \eta u) - \mathscr{L}(\theta) = u^T \nabla \mathscr{L}(\theta) = k * cos(\beta)$ will be most negative when $cos(\beta) = -1$ *i.e.*, when β is 180°

Gradient Descent Rule

- The direction u that we intend to move in should be at 180° w.r.t. the gradient
- In other words, move in a direction opposite to the gradient

$$\Delta\theta = u = -\nabla \mathscr{L}(\theta)$$

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Gradient Descent

Step 1. The function we want to minimize

- Let *v(x)* be a function (e.g., a loss function in machine learning).
- Goal: find *x* that minimizes *v(x)*.

Step 2. The gradient gives the direction of steepest increase

- Gradient:

$$\nabla v(x) = \left(\frac{\partial v}{\partial x_1}, \frac{\partial v}{\partial x_2}, \dots \right)$$

- Properties:
  - Points in the direction of **maximum increase** of *v*.
  - Magnitude shows how fast *v* increases.

Step 3. Directional derivative — effect of moving in any direction

- For a direction u (unit vector), the rate of change is:

$$D_u v(x) = u \cdot \nabla v(x)$$

- If $u \cdot \nabla v > 0$: v increases along u.
- If $u \cdot \nabla v < 0$: v decreases along u.

Step 4. Choosing the best direction — steepest descent

- Maximum decrease happens in the direction of the **negative gradient**:

$$u = -\frac{\nabla v(x)}{\|\nabla v(x)\|}$$

- This gives the steepest drop in v(x).

Step 5. Gradient descent update rule

- To update x step by step:

$$x_{\text{new}} = x_{\text{old}} - \eta \nabla v(x_{\text{old}})$$

Where:

  - ɳ learning rate (step size).

- Repeating this moves x gradually toward a local minimum.

Gradient Descent

We would like move in the direction of gradient only by a small step η(eta) to avoid overshooting the minimum value.

ηis also called the learning rate.

So final equations become:

$$w_{t+1} = w_t - \eta \nabla w_t$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

$$where, \nabla w_t = \frac{\partial \mathscr{L}(w,b)}{\partial w}_{at\ w=w_t, b=b_t}, \nabla b_t = \frac{\partial \mathscr{L}(w,b)}{\partial b}_{at\ w=w_t, b=b_t}$$

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Taylor series --- Recap

Algorithm 1: gradient_descent()

$t \leftarrow 0;$

$max_iterations \leftarrow 1000;$

while $t < max_iterations$ **do**

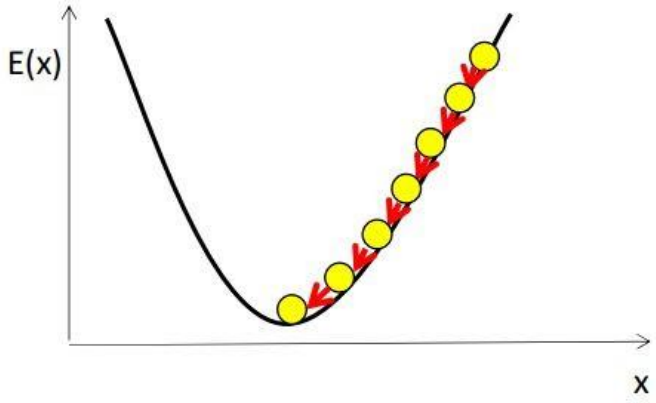
$w_{t+1} \leftarrow w_t - \eta \nabla w_t;$

$b_{t+1} \leftarrow b_t - \eta \nabla b_t;$

end

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

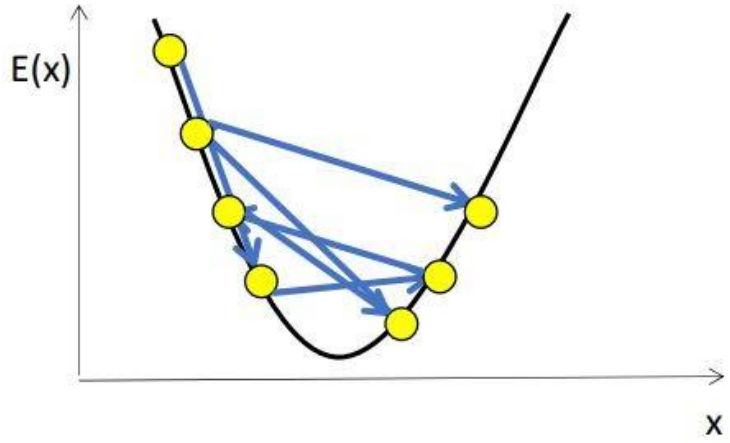
Learning rate being too small



$$w_{t+1} = w_t - \eta \nabla w_t$$

- Suppose we keep our η too small.
- Let's start with a point x.
- If η is too small on each updation we are changing our x with some small value i.e taking small steps.
- Again, from the new point, we take small steps to get to the minimum.
- If η is small, gradient descent can be slow to reach minimum.

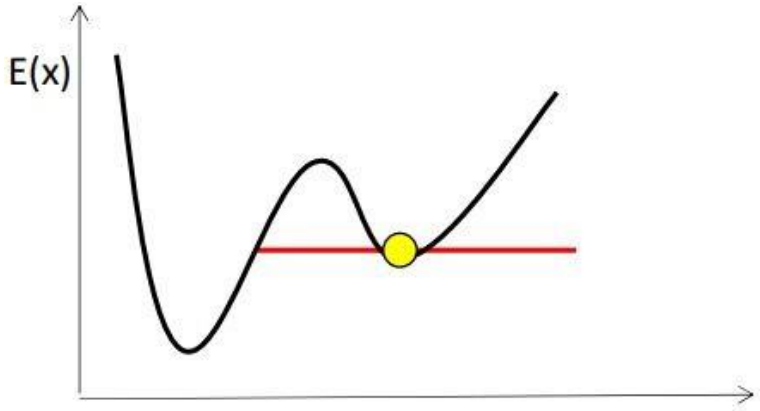
Learning rate being too large



$$w_{t+1} = w_t - \eta \nabla w_t$$

- If ɳs too large, then gradient may overshoot the minimum or may never reach minimum.
- Suppose we start at point x'.
- If ɳs too large, we take huge steps. Let's say we take this step and reach a new point.
- On the subsequent updation, we take a huge step and go right past the minimum.
- This may continue and we may never end up reaching the minimum.

Problem faced by gradient descent



$$w_{t+1} = w_t - \eta \nabla w_t$$

Suppose our x is already at local minimum, what would a gradient descent step do?

- At the local minimum, the slope would be zero. Hence, the derivative term is zero.
- It leaves our x unchanged.
- This is one problem with gradient descent – you may get stuck in a local minimum instead of reaching the global minimum.

Is there any ideal value for Learning rate???

Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1. 1.0]
- Run a few epochs with each of these and figure out a learning rate which works best
- Now do a finer search around this value [for example, if the best learning rate was 0.1 then now try some values around it: 0.05, 0.2, 0.3]
- Disclaimer: these are just heuristics ... no clear winner strategy

http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Is there any ideal value for Learning rate???

- As an alternative to constant learning, one could use adaptive learning rates(also called **learning rate annealing**) to increase performance and reduce training time.

- The simplest and perhaps most used adaptation of learning rate during training are techniques that reduce the learning rate over time. These have the benefit of making large changes at the beginning of the training procedure when larger learning rate values are used, and decreasing the learning rate such that a smaller rate and therefore smaller training updates are made to weights later in the training procedure.

- This has the effect of quickly learning good weights early and fine tuning them later.

Backward Propagation

Process of modifying the parameters (weights, bias) with an objective of minimizing the loss/error in the predicted value at each neuron in the output layer.

Backward propagation

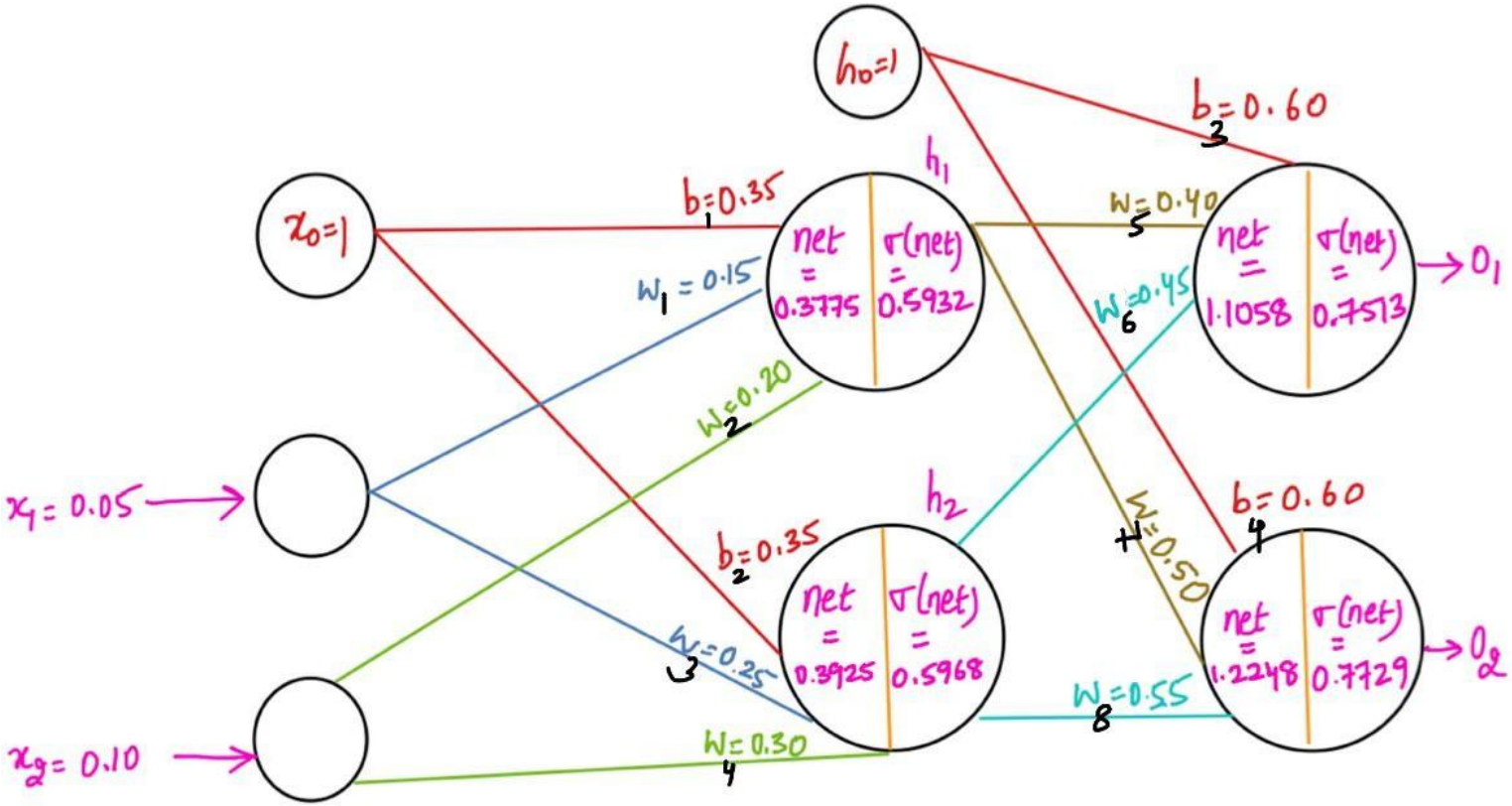
- How do we know whether the output generated by the neural network is correct?
  We need to define a **error function (cost)** and **backpropagate** the error.

- **Iteratively** process a set of training tuples and compare the network's prediction with the actual known target value.

- For each training tuple, the weights are modified to **minimize the loss function** between the network's prediction and the actual target value.

- Modifications are made in the "**backward**" direction i.e. from the output layer, through each hidden layer down to the first hidden layer, hence the name "**back propagation**".

Backward propagation



Neuron # in Output layer (k)	Target values at output neuron (t)	Predicted values (O)	Error in prediction (t_k - O_k)
1	0.01	0.7513	-0.7413
2	0.99	0.7729	0.2171

$$\text{MSE} = \frac{1}{2}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

$$E_d(\vec{w}) \equiv \frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2$$

Backward propagation -- Computing Error at Output layer

Neuron # in Output layer ($k \in$ outputs)	Target values at output neuron (t)	Predicted values (O)	Error in prediction ($t_k - O_k$)	$(t_k - O_k)^2$
1	0.01	0.7513	-0.7413	0.5495
2	0.99	0.7729	0.2171	0.0471
			$\sum_{k \in outputs} (t_k - o_k)^2$	0.5966
			$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$	0.2983

Backward Propagation

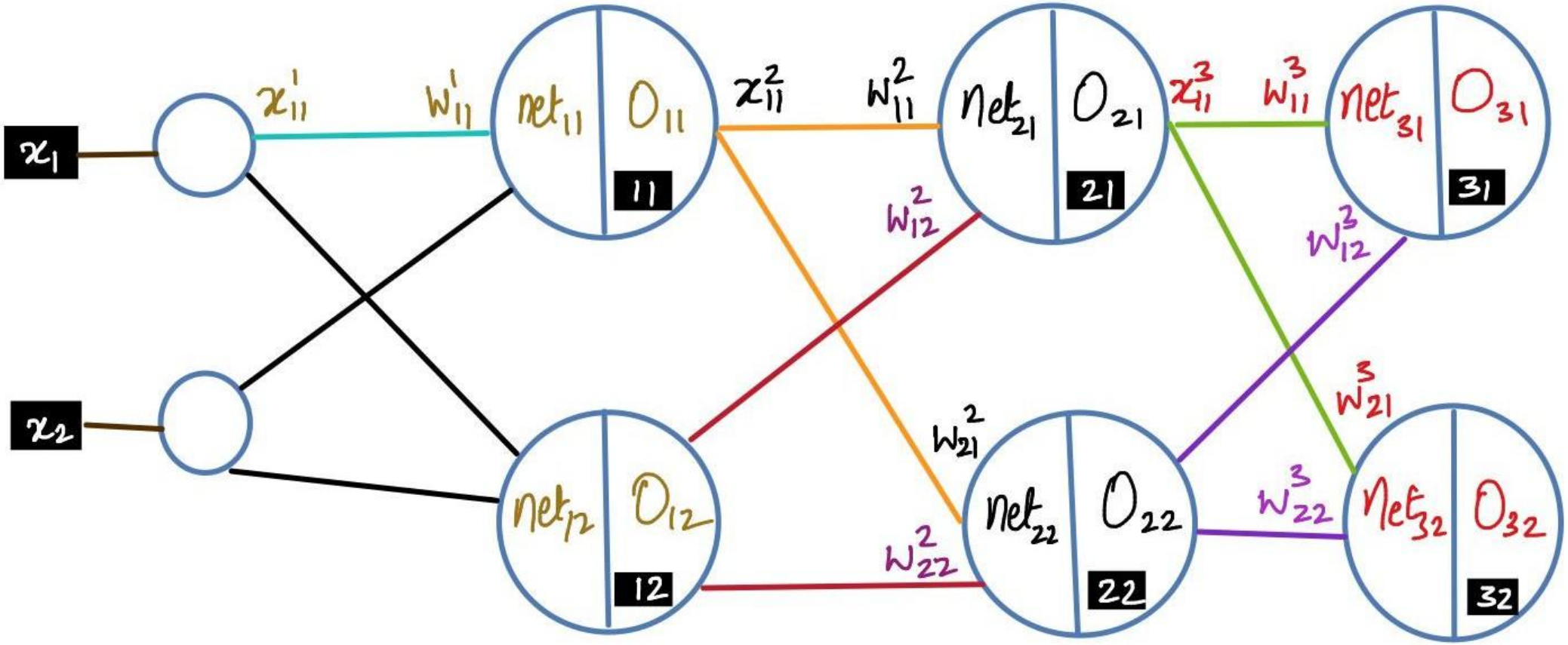
Understanding weight update with a Toy example

Understanding Weight update using a Toy example

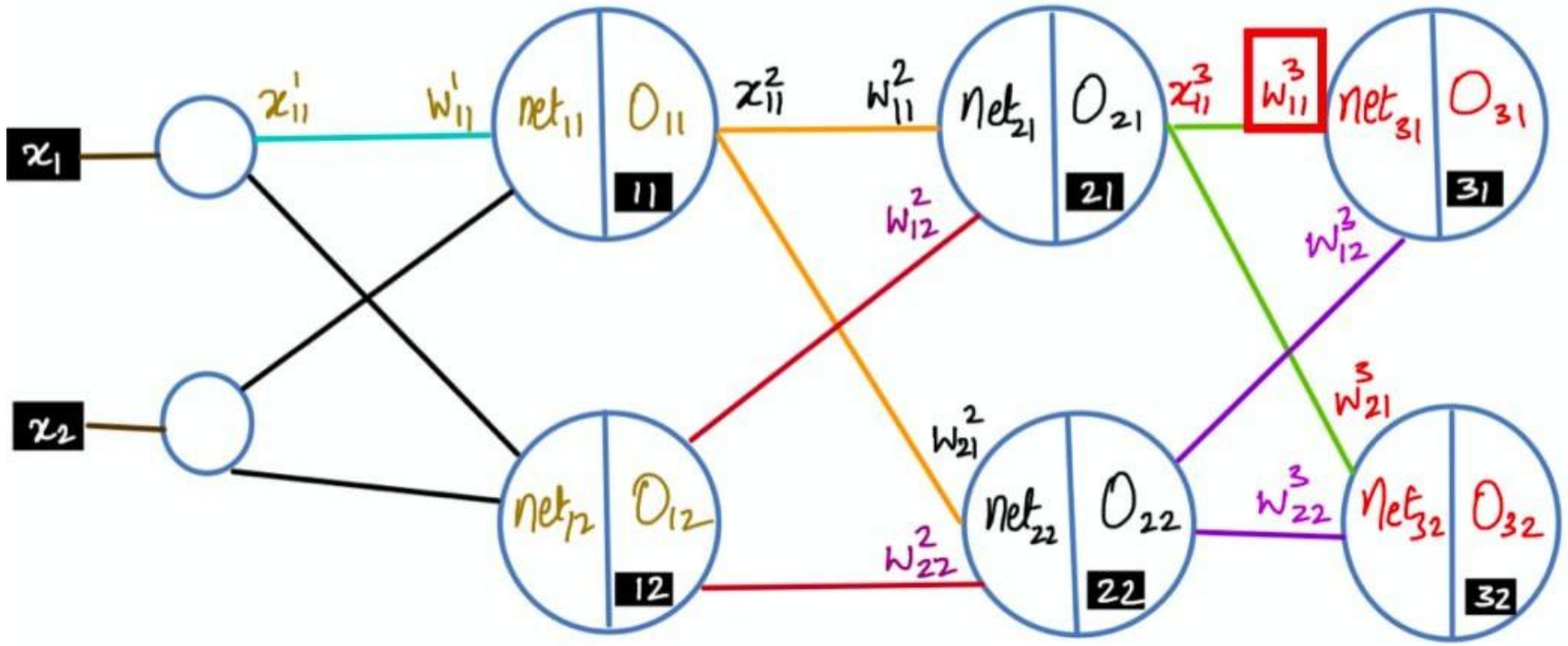
Weights are updated as using the gradient descent method.

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji} \quad \text{where}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$



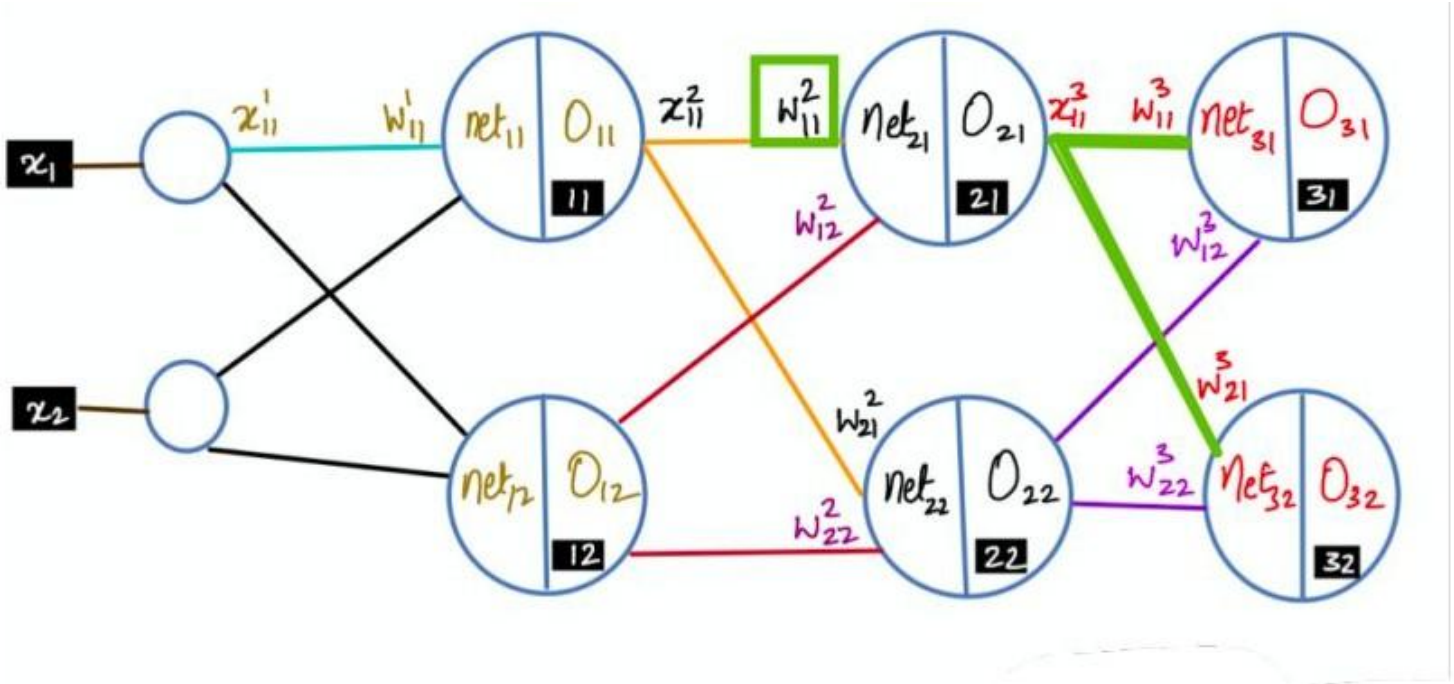
Updating Weight at Output layer



$$w_{11}^3 = w_{11}^3 + \Delta w_{11}^3 \;, \quad \Delta w_{11}^3 = -\eta \frac{\partial E_d}{\partial w_{11}^3}$$

$$\frac{\partial E_d}{\partial w_{11}^3} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \frac{\partial net_{31}}{\partial w_{11}^3}$$

Updating Weight at Hidden Layer 2

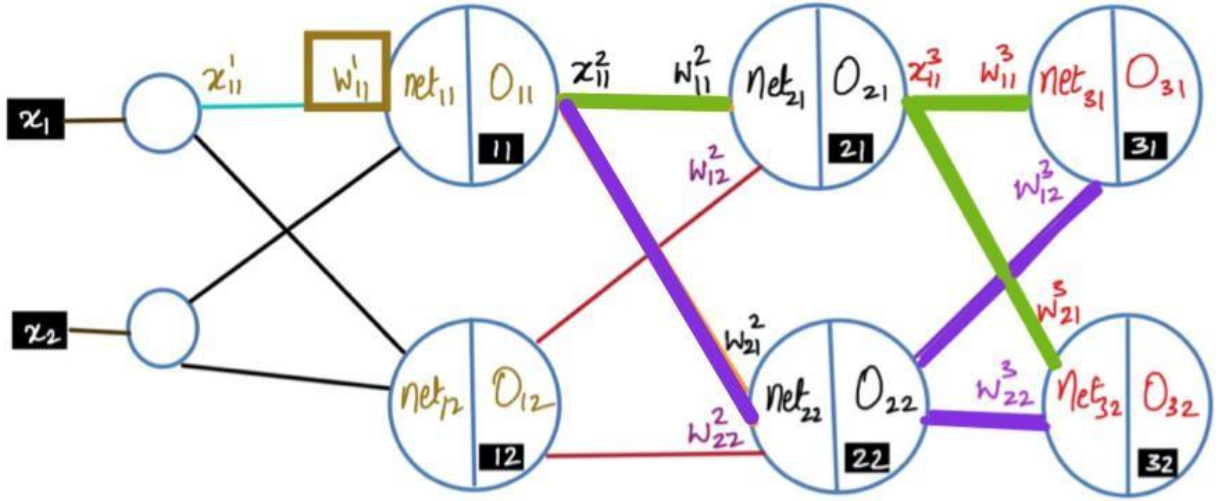


$$w_{11}^2 = w_{11}^2 + \Delta w_{11}^2,$$

$$\Delta w_{11}^2 = -\eta \frac{\partial E_d}{\partial w_{11}^2}$$

$$\frac{\partial E_d}{\partial w_{11}^2} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \frac{\partial net_{31}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial w_{11}^2} + \frac{\partial E_d}{\partial O_{32}} \frac{\partial O_{32}}{\partial net_{32}} \frac{\partial net_{32}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial w_{11}^2}$$

Updating Weight at Hidden Layer 1



$$w_{11}^1 = w_{11}^1 + \Delta w_{11}^1$$

$$\Delta w_{11}^1 = -\eta \frac{\partial E_d}{\partial w_{11}^1}$$

$$\frac{\partial E_d}{\partial w_{11}^1} = \frac{\partial E_d}{\partial O_{31}} \frac{\partial O_{31}}{\partial net_{31}} \left[\frac{\partial net_{31}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w_{11}^1} + \frac{\partial net_{31}}{\partial O_{22}} \frac{\partial O_{22}}{\partial net_{22}} \frac{\partial net_{22}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w_{11}^1} \right] + \frac{\partial E_d}{\partial O_{32}} \frac{\partial O_{32}}{\partial net_{32}} \left[\frac{\partial net_{32}}{\partial O_{21}} \frac{\partial O_{21}}{\partial net_{21}} \frac{\partial net_{21}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w_{11}^1} + \frac{\partial net_{32}}{\partial O_{22}} \frac{\partial O_{22}}{\partial net_{22}} \frac{\partial net_{22}}{\partial O_{11}} \frac{\partial O_{11}}{\partial net_{11}} \frac{\partial net_{11}}{\partial w_{11}^1} \right]$$

Backward Propagation Derivation Rules

Updating weights to lower the error

Error computation for a single training instance considering error at all neurons in output layer.

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

Weights are updated as using the gradient descent method.

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

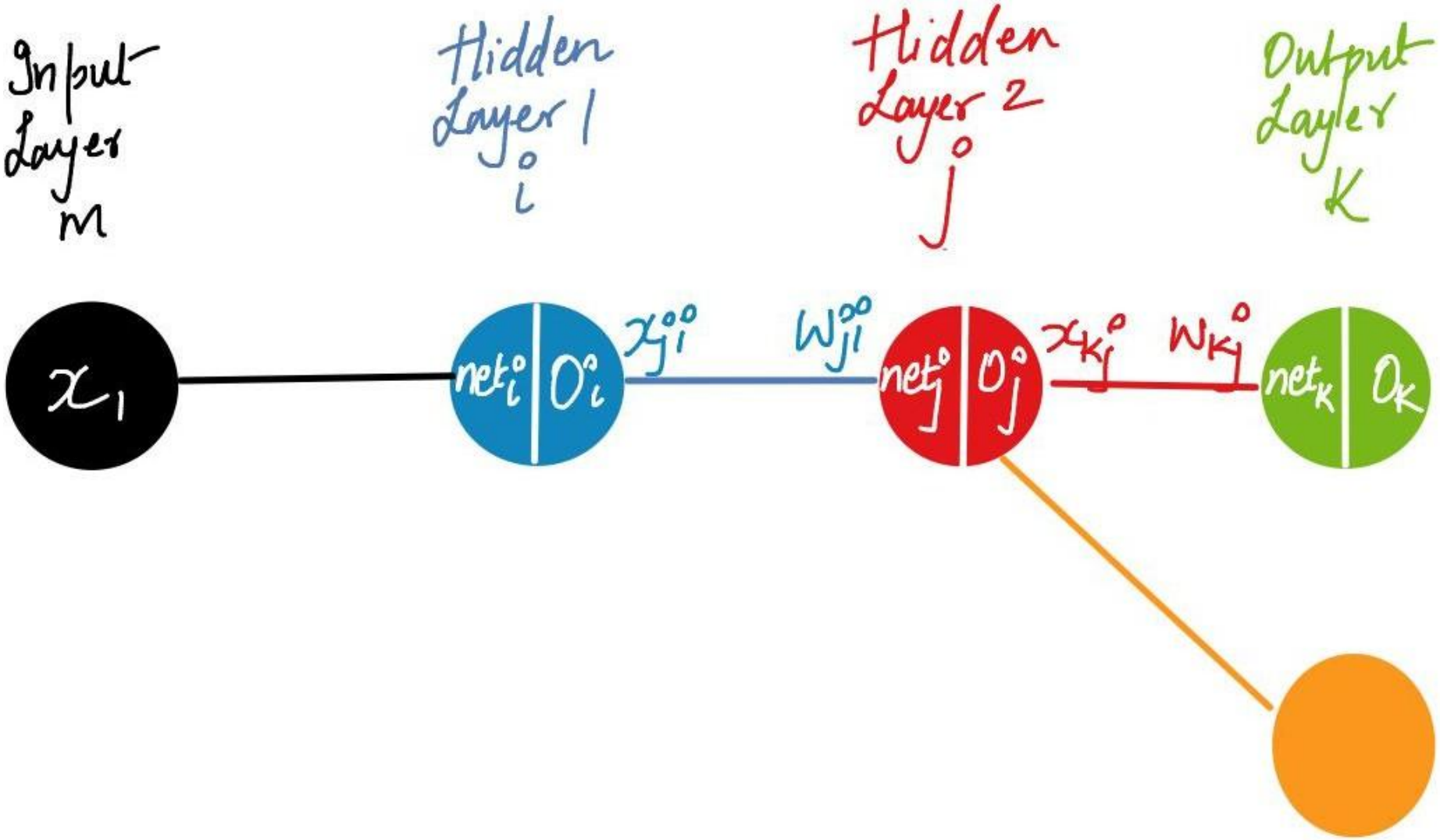
where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

Notice the weight can influence the rest of the network only through netj. Hence, we use the chain rule :

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}$$

Backward propagation

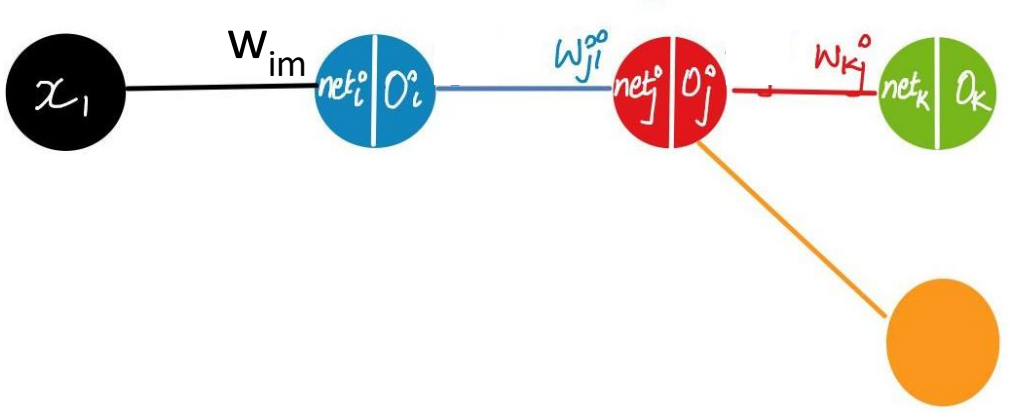


Training rule for output unit weights

$$w_{kj} = w_{kj} + \Delta w_{kj}$$

we know, $\Delta w_{kj} = -\eta \frac{\partial E_d}{\partial w_{kj}}$

$$\Rightarrow \Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial net_k} \times \frac{\partial net_k}{\partial w_{kj}}$$



$$net_k = \sum_j w_{kj} O_j$$

$$\boxed{\frac{\partial net_k}{\partial w_{kj}} = O_j}$$

Training rule for output unit weights

$$E_d = \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - O_k)^2$$

$$\boxed{\frac{\partial E_d}{\partial O_k} = -(t_k - O_k)}$$

$$O_k = \sigma(net_k)$$

$$\frac{\partial O_k}{\partial net_k} = \sigma(net_k)\,(1 - \sigma(net_k))$$

$$\boxed{\frac{\partial O_k}{\partial net_k} = O_k\,(1 - O_k)}$$

$$\Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial net_k} \times \frac{\partial net_k}{\partial w_{kj}}$$

$$y = \sigma(x) = \frac{1}{1+e^{-x}}$$

$$\frac{dy}{dx} = \frac{d\sigma(x)}{dx} = \frac{d}{dx}\left\{\frac{1}{1+e^{-x}}\right\}$$

$$= -(1+e^{-x})^{-2} \frac{d(1+e^{-x})}{dx}$$

$$= -(1+e^{-x})^{-2} \left(\frac{d(1)}{dx} + \frac{d(e^{-x})}{dx}\right)$$

$$= -(1+e^{-x})^{-2} \times -e^{-x}$$

$$= \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}}\left(1 - \frac{1}{1+e^{-x}}\right)$$

$$= \sigma(x)\,(1 - \sigma(x)) = y(1-y)$$

Training rule for output unit weights

$$\Delta w_{kj} = -\eta \frac{\partial E_d}{\partial O_k} \times \frac{\partial O_k}{\partial net_k} \times \frac{\partial net_k}{\partial w_{kj}}$$

$$\Delta w_{kj} = -\eta \left(-(t_k - O_k)\right) \times \left(O_k(1 - O_k)\right) \times (O_j)$$

$$\Delta w_{kj} = \eta (t_k - O_k)\, O_k(1 - O_k)\, O_j$$

$$\delta_k \leftarrow -\frac{\partial E_d}{\partial net_k}$$

$$\delta_k \leftarrow (t_k - O_k)\, O_k(1 - O_k)$$

$$\Delta w_{kj} = \eta\, \delta_k\, O_j$$

Training rule for hidden layer 2 weights

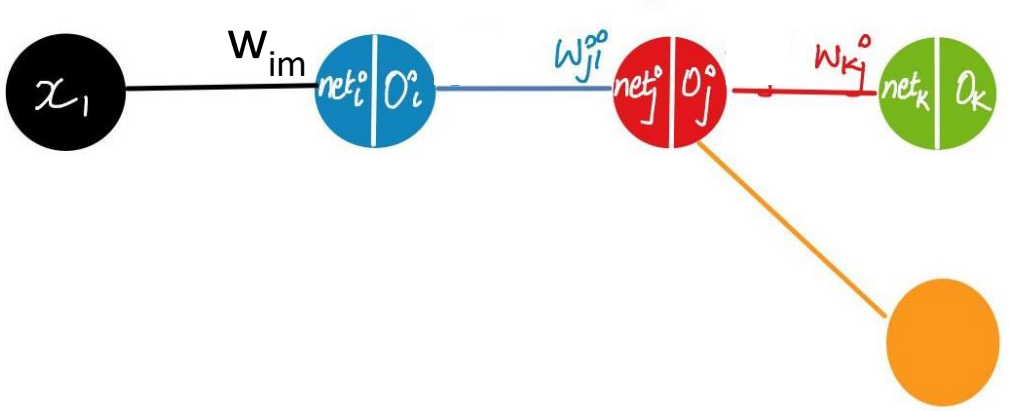
$$w_{ji} = w_{ji} + \Delta w_{ji}$$

we know, $\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$

$$\Rightarrow \Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} \times \frac{\partial net_j}{\partial w_{ji}}$$

$$net_j = \sum_v w_{ji} O_i$$

$$\boxed{\frac{\partial net_j}{\partial w_{ji}} = O_i}$$



Training rule for hidden layer 2 weights

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in downstream(j)} \frac{\partial E_d}{\partial O_k} \frac{\partial O_k}{\partial net_k} \frac{\partial net_k}{\partial O_j} \frac{\partial O_j}{\partial net_j}$$

$$net_k = \sum_j O_j w_{kj}$$

$$\frac{\partial net_k}{\partial O_j} = w_{kj}$$

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in down(j)} -\delta_k w_{kj} O_j (1 - O_j)$$

[where $\delta_k = (t_k - O_k)(O_k(1 - O_k))$]

Training rule for hidden layer 2 weights

$$\Delta w_{ji} = -\eta \times \boxed{\sum_{k \in down(j)} -\delta_k \, w_{kj} \, O_j (1 - O_j)} \times O_i$$

$$\Rightarrow -\delta_j$$

$$\boxed{\Delta w_{ji} = \eta \, \delta_j \, O_i}$$

where

$$\delta_j = \sum_{k \in downstream(j)} \delta_k \, w_{kj} \, O_j (1 - O_j)$$

$$\delta_k = (t_k - O_k) \, O_k (1 - O_k)$$

Training rule for hidden layer 1 weights

$$w_{im} = w_{im} + \Delta w_{im}$$

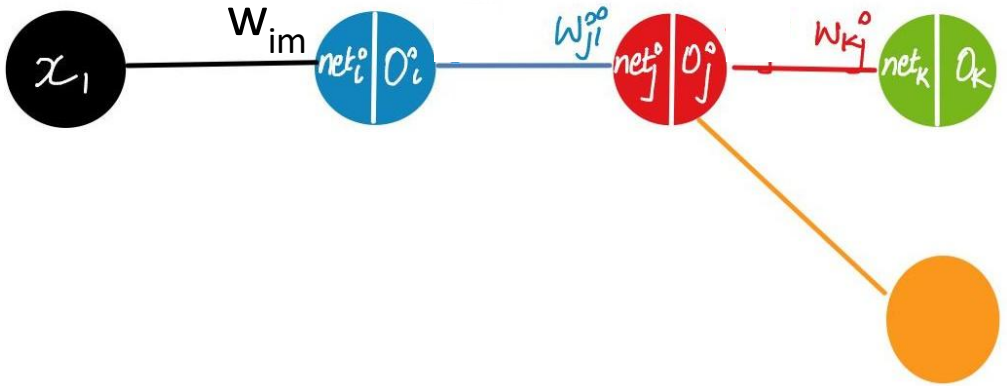
we know, $\Delta w_{im} = -\eta \frac{\partial E_d}{\partial w_{im}}$

$$\Rightarrow \Delta w_{im} = -\eta \frac{\partial E_d}{\partial net_i} \times \frac{\partial net_i}{\partial w_{im}}$$

$$net_i = \sum w_{im} x_m$$

$$\frac{\partial net_i}{\partial w_{im}} = x_m$$

$$\frac{\partial E_d}{\partial net_i} = \sum_{k \in downstream(i)} -\delta_i \; w_{ji} \; O_i (1 - O_i)$$



Training rule for hidden layer 1 weights

$$\Delta w_{im} = \eta \, \delta_i \, x_m$$

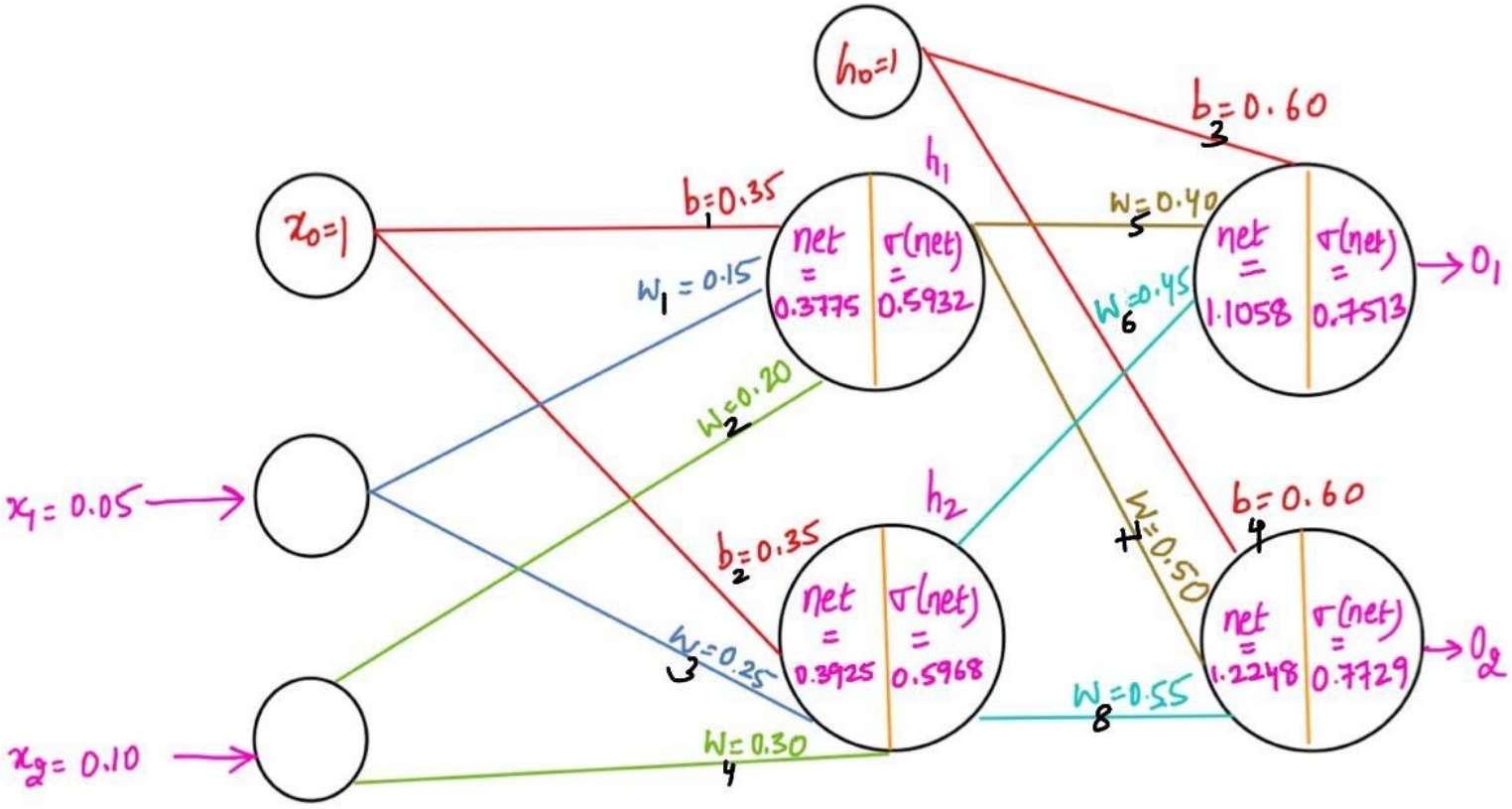
where $\delta_i = \sum_{k \in \text{downstream}(i)} \delta_j \, w_{ji} \, O_i (1 - O_i)$

$$\delta_j = \sum_{k \in \text{downstream}(j)} \delta_k \, w_{kj} \, O_j (1 - O_j)$$

$$\delta_k = (t_k - O_k) \, O_k (1 - O_k)$$

Backward Propagation Problem

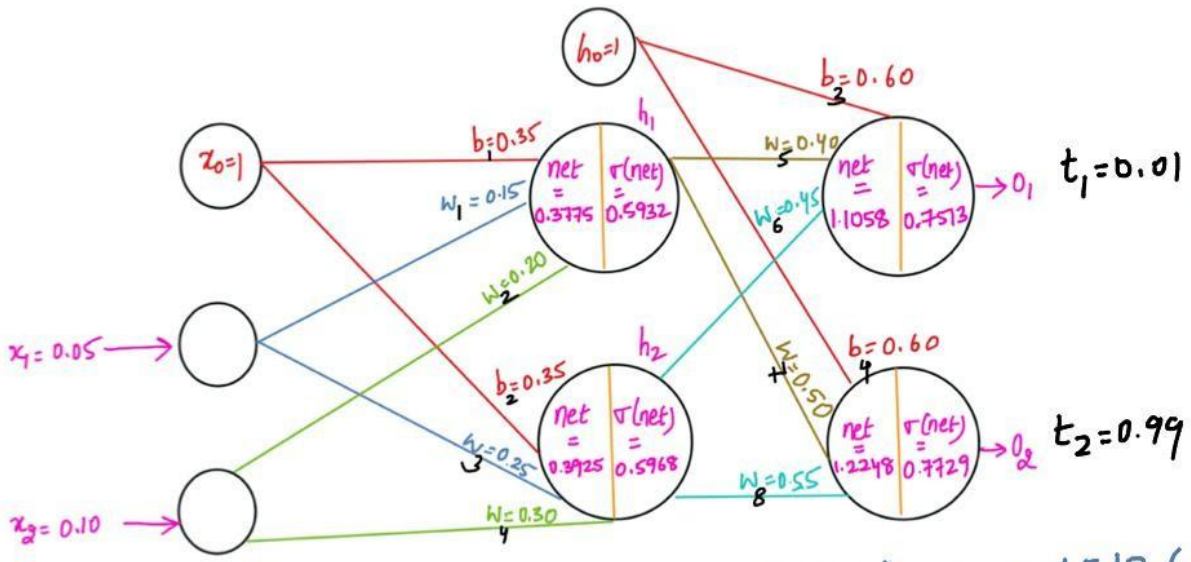
Using Backward propagation update w5, w7, w1



Neuron # in Output layer (k)	Target values at output neuron (t)	Predicted values (O)	Error in prediction (t_k - O_k)
1	0.01	0.7513	-0.7413
2	0.99	0.7729	0.2171

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

Updating w5 (output layer)



Output layer :-

$$\Delta w_{kj} = \eta * (t_k - O_k) * O_k(1-O_k) * x_{kj}$$

$$\Delta w_5 = 0.5 * (0.01 - 0.7513) * 0.7513(1-0.7513) * 0.5932$$

$$= 0.5 * -0.7413 * 0.7513 * 0.2487 * 0.5932$$

$$= 0.5 * -0.0821$$

$$= -0.04105$$

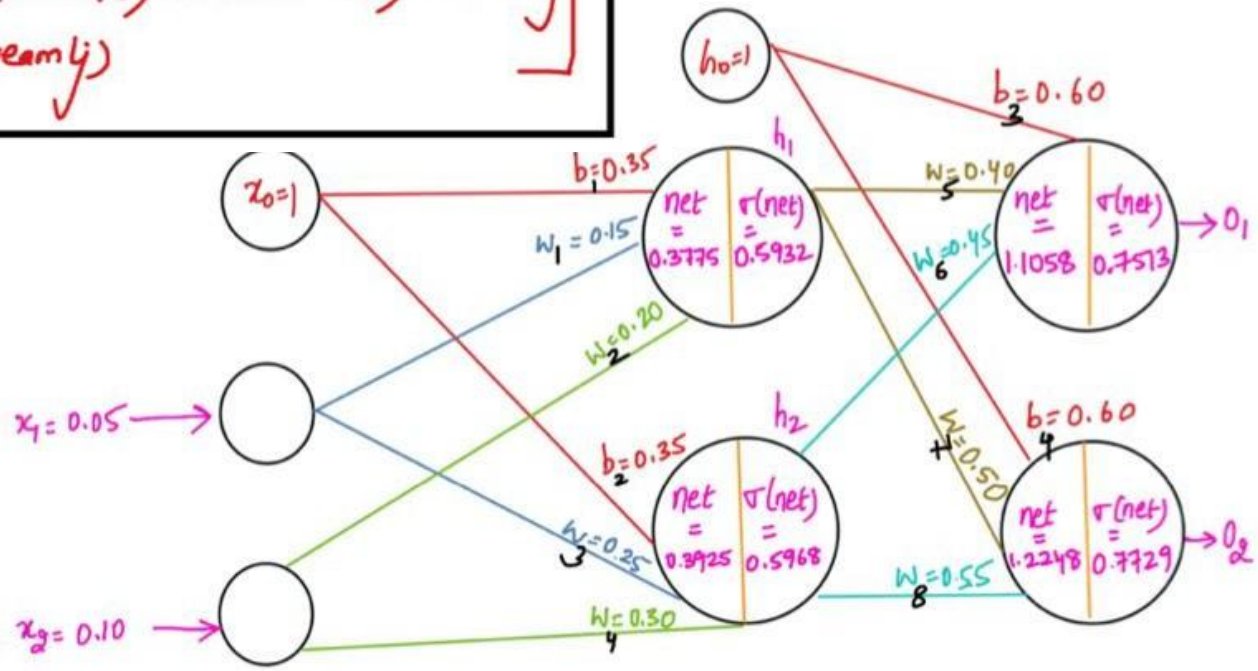
$$\Rightarrow w_5 = w_5 + \Delta w_5$$

$$= 0.40 + (-0.04105)$$

$$= 0.35895$$

Updating w1 (hidden layer)

$$\Delta w_{ji}^{o} = -\eta * O_j(1-O_j) * x_{ji}^{o} * \left[\sum_{k \in \text{Downstream}(j)} -(t_k - O_k) * O_k(1-O_k) * w_{kj}^{o}\right]$$



$$\Delta w_1 = -0.5 * 0.5932(1-0.5932) * 0.05 * \left[\begin{array}{l} -(0.01-0.7513) \\ * 0.7513(1-0.7513) * 0.40 \\ + \\ -(0.99-0.7729) \\ * 0.7729(1-0.7729) * 0.50 \end{array}\right]$$

$$= -0.00021$$

Vanishing Gradients Problem with Sigmoid

Vanishing Gradients

- As the backpropagation algorithm advances downwards (or backward) from the output layer towards the input layer, the **gradients often get smaller and smaller and approach zero** which eventually leaves the weights of the initial or lower layers nearly unchanged.

- As a result, the gradient descent never converges to the optimum. This is known as the *vanishing gradients* problem.

Overfitting – Analogous to Decision Trees case

- With several weights to update over several iterations, the backpropagation algorithm tends to overfit.
- Overfitting occurs because the weights are being tuned to fit noise of the training examples that are not representative of the general distribution of examples.

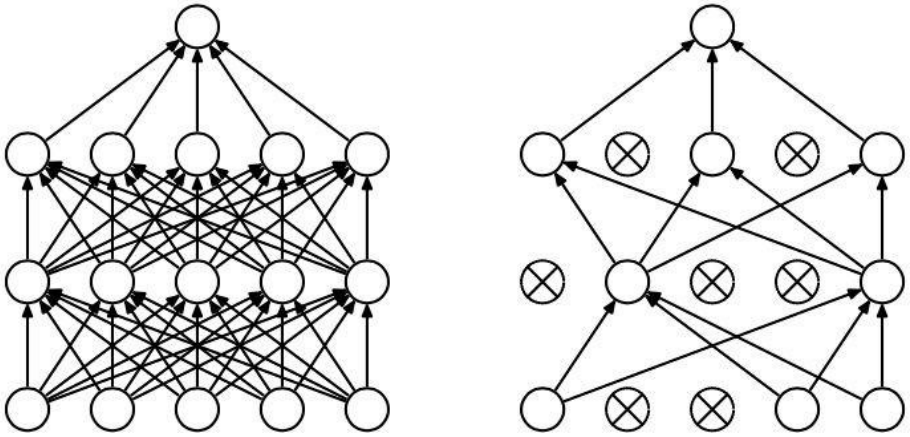
Possible solutions:

- Maintain a separate validation set, apart from training and test sets.
- Measure the accuracy over the validation set at intervals.
- Choose the set of weights that produce the least error on the validation set.

Overfitting – Some methods to avoid overfitting in Neural Networks

Dropout:

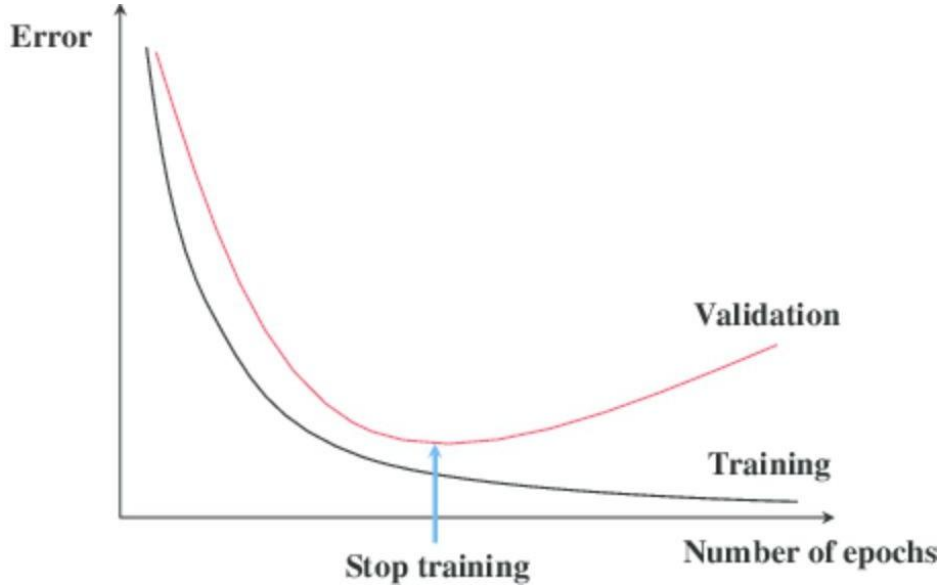
- It is a regularization technique that prevents the neural network from overfitting.
- It randomly drops neurons from the neural network during training which is equivalent to training different neural networks.
- The different networks will overfit differently, so the net effect of the dropout regularization technique will be to reduce overfitting so that our model will be good for predictive analysis.



Overfitting – Some methods to avoid overfitting in Neural Networks

Early termination:

- This regularization technique updates the model to make it better fit the training data with each iteration.
- After a certain number of iterations, new iterations improve the model.
- After that point, however, the model begins to overfit the training data.
- Early stopping refers to stopping the training process before that point.



Practice Questions

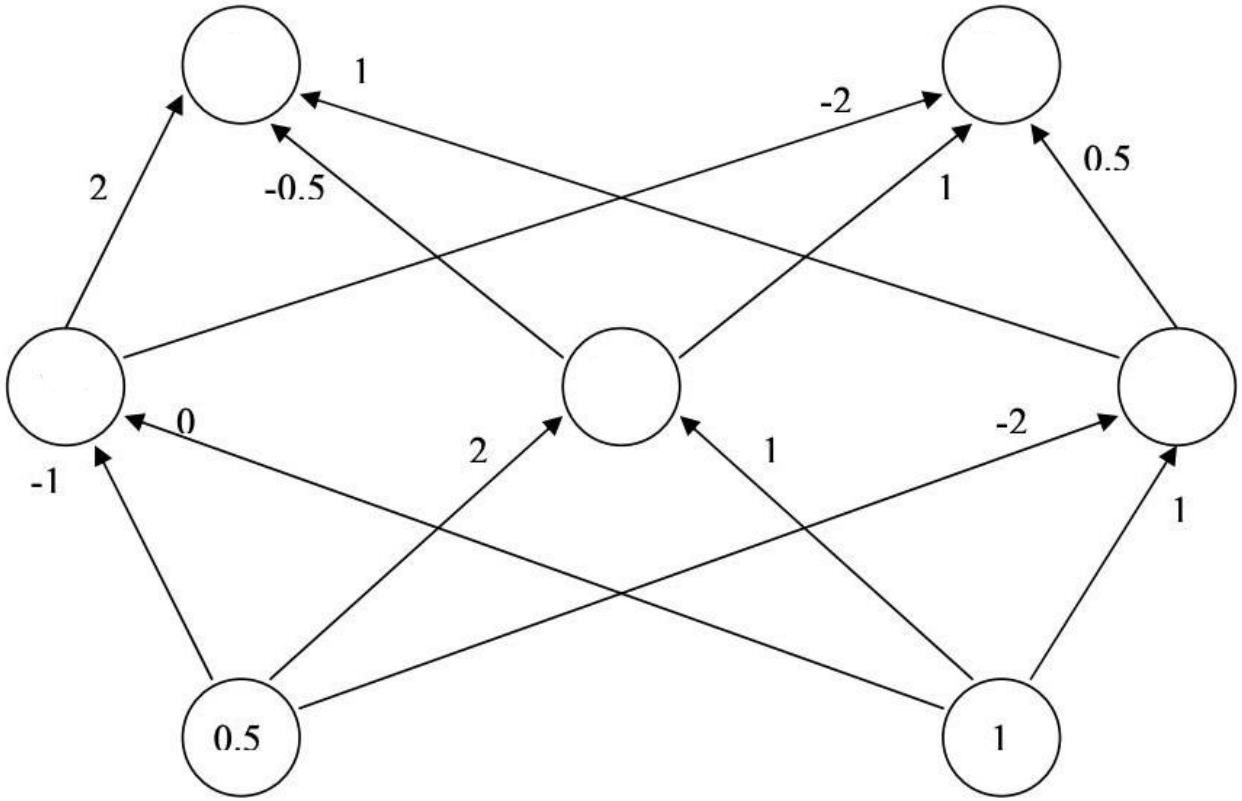
Practice Questions

Question on Linear Neurons:

The following is a network of linear neurons, that is, neurons whose output is identical to their net input, oi = neti. The numbers in the circles indicate the output of a neuron, and the numbers at connections indicate the value of the corresponding weight.

a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.

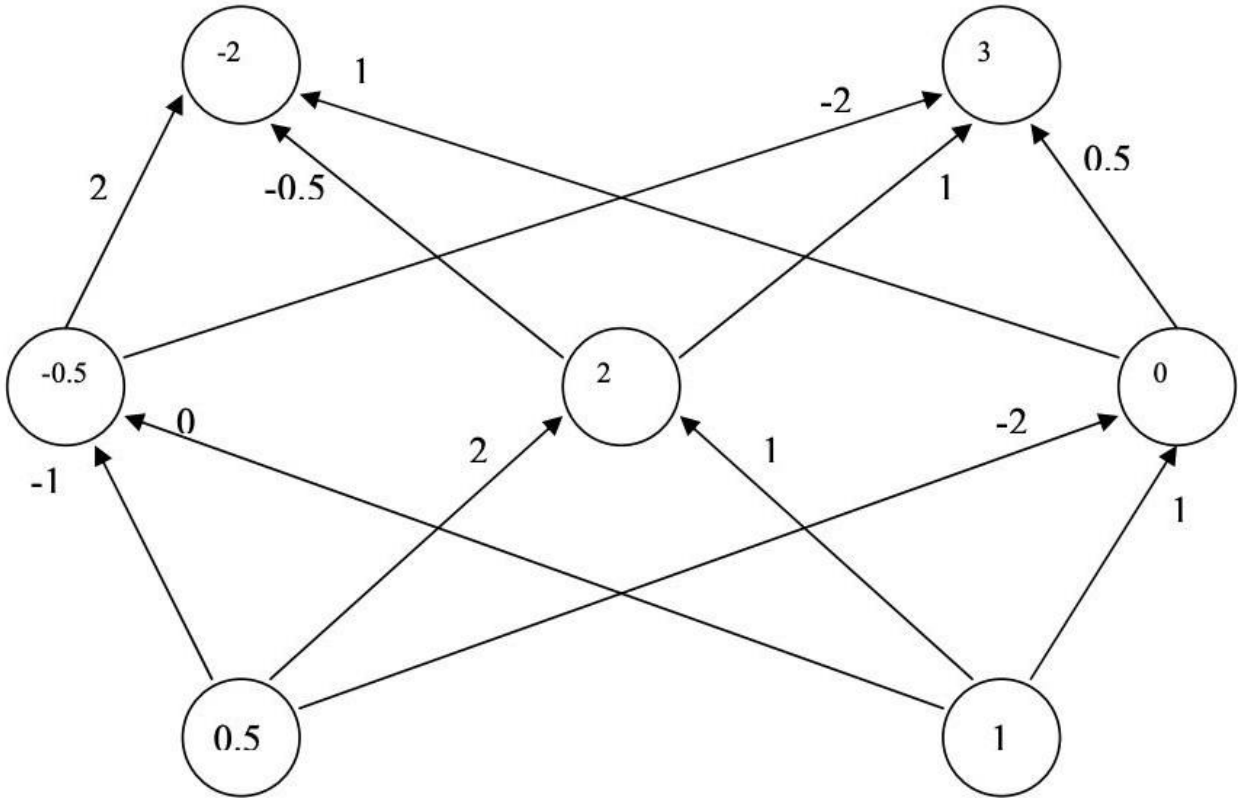
b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.



Practice Questions

a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles.
Result: (-2, 3).

b) What is the output of the network for the input (1, 2), i.e. the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this.
It's (-4, 6). We do not have to do all the computations, because every neuron computes a linear function on its inputs, which means that the entire network computes a linear function. For such a function, if we double the input, we simply double the output as well.



Practice Questions

A perceptron with a unipolar step function has two inputs with weights $w_1 = 0.5$ and $w_2 = -0.2$, and a threshold $\theta = 0.3$ (θ can therefore be considered as a weight for an extra input which is always set to -1). For a given training example $\mathbf{x} = [0, 1]^T$, the desired output is 0 (zero). Does the perceptron give the correct answer (that is, is the actual output the same as the desired output)?

A. Yes.

B. No.

Unipolar or

Binary step function:- This function can be defined as,

$$f(x) = \begin{cases} 1, & if \quad x \geq \theta \\ 0, & if \quad x < \theta \end{cases}$$

where θ represents the threshold value. This function is most widely used in single layer nets to convert the net input to an output that is binary (1 or 0).

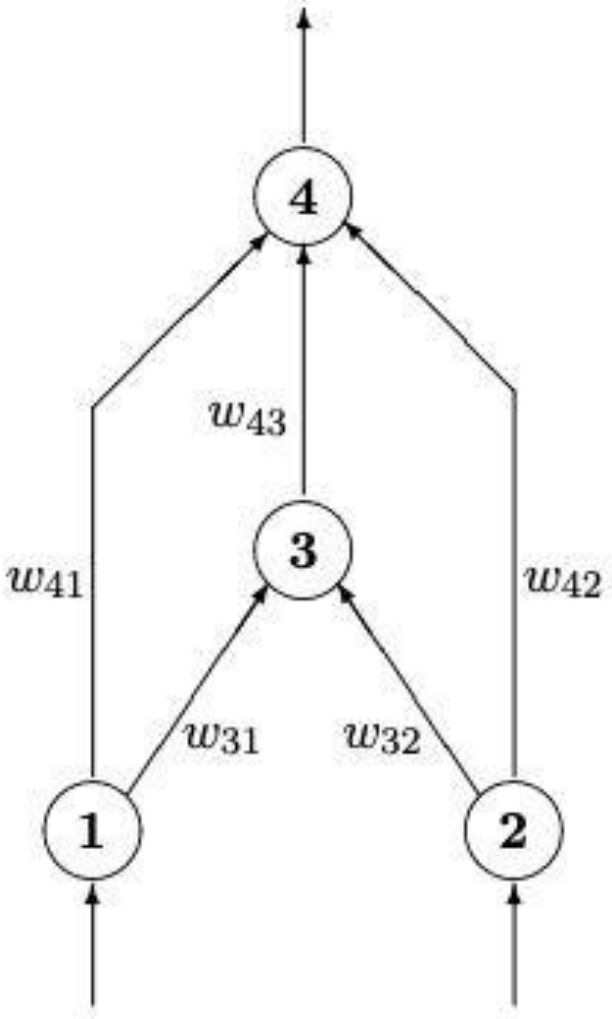
Practice Questions

The following network is a multi-layer perceptron, where all of the units have binary inputs (0 or 1) and binary outputs (0 or 1).

The weights for this network are w31 = 1, w32 = 1, w41 = −1, w42 = −1 and w43 = 3.

The threshold of the hidden unit (3) is 1.5 and the threshold of the output unit (4) is −0.5.

The threshold of both input units (1 and 2) is 0.5, so the output of these units is exactly the same as the input. Which of the following Boolean functions can be computed by this network?
AND, OR, XOR or none?



References

Sigmoid Neuron, Gradient Descent : http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture3.pdf

Backpropagation : https://www.youtube.com/watch?v=XN5IRqtFhOY

Numerical : https://www.youtube.com/watch?v=0e0z28wAWfg



UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu